AUTOML-DRIVEN PREDICTIVE ANALYSIS FOR INSIGHTS FROM ANALYTIC SURVEY DATA

A PROJECT REPORT

Submitted by

RAMYA S (2021503545)

RUPESH A (2021503549)

AKILAN K (2021503719)

in partial fulfilment for the award of the degree

of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE ENGINEERING



DEPARTMENT OF COMPUTER TECHNOLOGY

ANNA UNIVERSITY: CHENNAI 600 025

MAY 2025

ANNA UNIVERSITY : CHENNAI 600 025

BONAFIDE CERTIFICATE

Certified that this project report "**AUTOML-DRIVEN PREDICTIVE ANALYSIS FOR INSIGHTS FROM ANALYTIC SURVEY DATA"** is the bonafide work of **RAMYA S (2021503545), RUPESH A (2021503549) AND AKILAN K (2021503719)** who carried out the project work under my supervision.

SIGNATURE

Dr. P. PABITHA

SUPERVISOR

Associate Professor

Dept. of Computer Technology

Anna University, MIT Campus

Chromepet, Chennai 600 044

SIGNATURE

Dr. P. JAYASHREE

HEAD OF THE DEPARTMENT

Professor

Dept. of Computer Technology

Anna University, MIT Campus

Chromepet, Chennai 600 044

ABSTRACT

Machine Learning and Artificial Intelligence have become key technologies in solving real-world problems, such as recommendation systems, customer behavior prediction, and fraud detection. These systems learn from large amounts of data to find patterns and make smart decisions.

However, creating an effective ML solution involves several complex steps—data collection, cleaning, analysis, model selection, and fine-tuning—which can take a lot of time and usually require experts in data science. This makes it difficult for smaller teams or non-experts to fully use the power of ML.

To address this, Automated Machine Learning (AutoML) was introduced to simplify the process by automating tasks like model training and selection. While AutoML tools are helpful, they still have some key limitations. Most tools are designed mainly for structured (tabular) data and struggle to handle mixed or complex data types like images and text together.

Additionally, they often work like "black boxes," meaning users don’t get much insight into how the system makes decisions. This lack of transparency and flexibility makes it harder to trust or customize the system, especially when dealing with real-time or changing data.

Our project focuses on solving these challenges by building a semi-automated ML platform that streamlines the early stages of data processing. It supports uploading datasets in common formats like CSV and Excel, and also handles image-based data. The system automatically cleans and standardizes the data, then generates interactive exploratory data analysis (EDA) reports to help users understand key insights.

Our project reduces manual effort by automating up to 80% of early data processing tasks, enhancing accuracy and consistency. The system supports structured tabular data and unstructured image datasets, enabling multi-modal learning with over 30,000 image-based records and 250 structured sales transactions for analysis. By integrating PyCaret's model comparison, the framework efficiently evaluates 40 machine learning models, selecting the best-performing one based on metrics like accuracy, precision, and F1-score. This automation significantly accelerates ML workflows, reducing preprocessing time by at least 50%, while laying the groundwork for scalable, adaptive pipelines.

ACKNOWLEDGEMENT

We take this humble opportunity to thank Dean, MIT Campus, Anna University, **Dr. Ravichandran Kandaswamy** and **Dr. Jayashree P**, Professor & Head, Department of Computer Technology, MIT Campus, Anna University for providing all the lab facilities in pursuit of this project.

We would like to thank our supervisor Dr. Pabitha P, Associate Professor, Department of Computer Technology, Anna University, MIT Campus, whose kind guidance and proper directions helped shape this project perfectly. Constant encouragement to implement the project to the fullest of our potential propelled us towards the successful completion of the proposed project work.

We acknowledge and thank the efforts and feedback of the panel members **Dr. S Chitra**, Associate Professor, Department of Computer Technology, MIT, Anna University, **Dr.V P Jayachitra**, Assistant Professor, Department of Computer Technology, MIT, Anna University, and **Dr. K Kottilingam**, Assistant Professor, Department of Computer Technology, MIT, Anna University, in reviewing our work, providing constant valuable comments and encouraging us to view the different aspects of the project in successful implementation of the project.

We also thank all the faculty members of the Department of Computer Technology for their constant support and suggestions.

Ramya S (2021503545)

Rupesh A (2021503549)

Akilan K (2021503719)

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	**iv**
	LIST OF TABLES	**x**
	LIST OT FIGURES	**xi**
	LIST OF ABBREVATIONS	**xiii**
1.	**INTROUDUCTION**	**1**
	1.1 OVERVIEW	1
	1.2 SOCIAL NETWORK ANALYSIS	1
	1.3 MACHINE LEARNING	2
	1.4 AUTOMATED MACHINE LEARNING	3
	1.5 PREDICTIVE ANALYSIS	3
	1.6 EXPLORATORY DATA ANALYSIS	4
	1.7 AUTOMATED MODEL SELECTION	5
	1.8 ENSEMBLE LEARNING TECHNIQUES	6
	1.9 CROSS VALIDATION AND EVALUATION	7
	1.10 OBJECTIVES	8
	1.8 ORGANIZATION OF THESIS	8
2.	**LITERATURE SURVEY**	**10**
	2.1 OVERVIEW	10
	2.2 MULTI-MODEL / ADAPTIVE MODEL SELECTION	10

2.3 AUTOML AND DRL FOR OPTIMIZATION 12

2.4 META-LEARNING & RAPID ADAPTATION 13

2.5 LLM LIFECYCLE & OPERATIONS 14

2.6 INCREMENTAL/ CONTINUAL LEARNING 14

2.7 ADAPTIVE REPRESENTATION AND PREDICTION 16

2.8 DATA STREAM PROCESSING 17

2.9 SUMMARY 24

3. PROPOSED WORK 25

3.1 INTRODUCTION 25

3.2 PROPOSED WORK 25

3.3 WORKING PROCESS OF THE PROPOSED WORK 27

3.3.1 Data Ingestion and Data Preprocessing 28

3.3.2 Pseudo Code for Data ingestion and data preprocessing Module 31

3.3.3 Model Training and Selection 32

3.3.4 Pseudo Code for Model Training and Selection 34

3.3.5 Model Comparison 35

3.3.6 Pseudo Code for Model Comparison Module 37

3.4 SUMMARY 38

4. IMPLEMENTATION 39

4.1 TOOLS REQUIRED 39

4.1.1 pandas (Data handling, manipulation) 39

4.1.2 os (File management, directory control) 40

4.1.3 Numpy (Array operations, numerical processing) 40

4.1.4 joblib (Model saving, caching) 41

4.1.5 ydata profiling (Automated data insights) 41

4.1.6 scikit-learn (Machine learning, preprocessing) 41

4.1.7 pycaret (AutoML, model tuning) 42

4.1.8 matplotlib (visual analysis) 42

4.1.9 seaborn (pattern discovery) 43

4.2 DATASET DESCRIPTION 45

4.2.1 Text dataset description 45

4.2.2 Image dataset description 46

4.3 DATA INGESTION AND DATA PREPROCESSING MODULE 47

4.3.1 Implementation of Data ingestion and data preprocessing Module 47

4.3.2 Code for Dataset Uploading And Eda Report Generation 48

4.4 MODEL TRAINING AND SELECTION MODULE 49

4.4.1 Implementation of Model Training and Selection Module 50

4.4.2 Code for Model Training and Selection Module 51

4.5 MODEL COMPARISON 54

4.5.1 Implementation of Model comparison Module 55

4.5.2 Code for Model comparison Module 56

4.6 SUMMARY 58

5. RESULTS AND ANALYSIS 59

5.1 OBSERVATIONS 59

5.1.1 Uploading dataset 59

5.1.2 Exploratory data analysis (EDA) generation 60

5.1.3 Model training and selecting best model 65

5.1.4 Hyperparameter Model Tuning 68

5.2 INTERPRETATION 70

5.2.1 Evaluated Machine learning models 71

5.2.2 Model comparison based on Performance metrics 71

5.3 SUMMARY 77

6. CONCLUSION AND FUTURE WORK 78

6.1 CONCLUSION 78

6.2 FUTURE WORK 78

REFERENCES 79

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
2.1	SUMMARY OF AUTOML APPROACHES	18
4.1	LIST OF LIBRARIES USED	44
4.2	FEATURES AND DESCRIPTION OF TEXT DATASET	45
4.3	FEATURES AND DESCRIPTION OF IMAGE DATASET	46

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
3.1	BLOCK DIAGRAM OF AUTOML-DRIVEN INSIGHTS FROM SURVEY DATA	27
3.2	WORK FLOW OF DATA INGESTION AND DATA PREPROCESSING MODULE	30
3.3	WORK FLOW OF MODEL TRAINING AND SELECTION MODULE	34
5.1	DATASET UPLOADED USING STREAMLIT APP FOR MODEL SELECTION	59
5.2	EXPLORATORY DATA ANALYSIS REPORT FOR THE DATASET	60
5.3	EDA REPORT FOR EACH PARAMETER IN THE DATASET	61
5.4	INTERACTION GRAPH BETWEEN THE PARAMETERS TOTAL SALES AND PRICE	62
5.5	CONFUSION MATRIX FOR THE FEATURES IN THE DATASET	63
5.6	TRAINING MODEL FOR THE DATASET	65
5.7	TRAINING MODEL FOR THE GIVEN DATASET	66
5.8	TRAINING MODELS (DECISION TREE, GRADIENT BOOSTING)	66
5.9	DISPLAYING THE BEST MODEL BASED ON ANALYSIS	67

5.10	PERFORMANCE REPORT FOR THE MODELS	68
5.11	HYPERPARAMETERS TUNING FOR THE MODELS	69
5.12	MODEL COMPARISON USING BAR GRAPH	70
5.13	MODEL COMPARISON BASED ON ACCURACY	72
5.14	MODEL COMPARISON BASED ON AUC	73
5.15	MODEL COMPARISON BASED ON RECALL	73
5.16	MODEL COMPARISON BASED ON PRECISION	74
5.17	MODEL COMPARISON BASED ON F1-SCORE	74
5.18	MODEL COMPARISON BASED ON KAPPA	75
5.19	MODEL COMPARISON BASED ON MCC	75
5.20	MODEL COMPARISON BASED ON TRAINING TIME(TT)	76

LIST OF ABBREVIATIONS

SNA	Social Network Analysis
ML	Machine Learning
AI	Artificial Intelligence
EDA	Exploratory Data Analysis
DQN	Deep Q-learning Network
LLM	Large Language Model
ETC	Explore-Then-Commit
AUC	Area Under the Curve
TT	Training Time
MCC	Matthews Correlation Coefficient

CHAPTER 1

INTRODUCTION

1.1 OVERVIEW

The introduction sets the stage for understanding automation of machine learning workflows to enhance predictive analysis of survey data. It addresses challenges in manual data preprocessing, exploratory data analysis (EDA), and model selection. By integrating AutoML, it simplifies data ingestion, standardization, and analysis while improving efficiency. The system generates automated EDA reports and selects optimal models, making data-driven decision-making more accessible. Ultimately, it enhances model adaptability, transparency, and scalability for better analytical insights.

1.2 SOCIAL NETWORK ANALYSIS

Social network analysis (SNA) is a technique used to study relationships between individuals, groups, or systems through network structures. It plays a vital role in various fields, including sociology, business analytics, and AI, enabling better understanding of interactions within social and technological networks. SNA often leverages graph theory to analyze patterns, influence, and connectivity among entities.

One major aspect of SNA is detecting influential nodes and communities within networks. Metrics such as degree centrality, betweenness centrality, and clustering coefficients help quantify the importance of individual nodes. These insights can be applied to areas like marketing strategies and political influence modeling.

Integrating SNA with machine learning allows predictive insights into evolving network structures. For example, AI-driven models can forecast network behavior, detect anomalies, and optimize engagement strategies. In the context of your project, SNA can assist in analyzing survey data to uncover trends in user interactions and behaviors.

1.3 MACHINE LEARNING

Machine learning (ML) is a field of artificial intelligence (AI) that enables computers to learn from data and make predictions or decisions without being explicitly programmed. It involves various techniques such as supervised learning, where models are trained on labeled data, and unsupervised learning, where patterns are detected in unlabeled data. The rise of ML has transformed industries, powering recommendation systems, fraud detection, and predictive analytics.

The ML workflow includes data preprocessing, feature selection, model training, and evaluation. Each step is crucial for building robust models that generalize well to new data. Traditional approaches require expert knowledge to select the best algorithms and fine-tune hyperparameters, which can be time-consuming and challenging.

AutoML seeks to streamline this process by automating many aspects of ML, including model selection and optimization. It enables non-experts to apply ML techniques efficiently while ensuring high performance. However, AutoML still faces challenges, such as handling complex datasets involving images, text, or multi-modal data. Addressing these limitations is essential to make the AutoML that are more adaptable, capable and transparent.

1.4 AUTOMATED MACHINE LEARNING (AutoML)

Automated Machine Learning (AutoML) is a rapidly evolving field that aims to simplify the machine learning process by automating key tasks such as data preprocessing, feature engineering, model selection, and hyperparameter tuning. Traditional ML workflows require extensive domain expertise and trial-and-error experimentation, but AutoML democratizes access to predictive modeling by making advanced techniques more efficient and user-friendly.

AutoML platforms leverage algorithms to search for the best model architecture, optimize performance, and reduce human intervention. Popular AutoML frameworks include PyCaret, H2O.ai, Auto-sklearn, and Google AutoML. These systems analyze multiple models, compare their effectiveness, and select the most suitable one based on predefined criteria such as accuracy, precision, and recall. This is particularly beneficial for projects involving large-scale datasets, as it accelerates the decision-making process while ensuring high-performing results.

Despite its advantages, AutoML faces challenges, including handling complex, multi-modal datasets (such as text, images, and audio) and maintaining interpretability. In your project, AutoML is integrated to assist in predictive analytics for survey data insights, automatically selecting and training models for optimal results. The next frontier for AutoML involves improving adaptability to real-time learning environments and enhancing transparency in automated decision-making.

1.5 PREDICTIVE ANALYSIS

Predictive analytics applies statistical and machine learning techniques to forecast future trends based on historical data. It is widely used in domains such as healthcare, finance, and marketing to anticipate customer behavior, detect fraudulent activities committed, and also optimize decision-making.

The process involves collecting and cleaning data, selecting relevant features, and applying models like regression, classification, and clustering. Advanced predictive models leverage deep learning for more nuanced pattern recognition, making them powerful tools for business intelligence and automation.

In this project, predictive analytics plays a crucial role in extracting valuable insights from survey data. AutoML enhances this by automating feature selection, model tuning, and evaluation, making predictive capabilities more accessible. The challenge lies in ensuring models remain interpretable while adapting to dynamic data environments.

1.6 EXPLORATORY DATA ANALYSIS (EDA)

Exploratory Data Analysis (EDA) is a crucial step in data science and machine learning that involves examining and understanding a dataset before applying any modeling techniques. It helps uncover patterns, anomalies, correlations, and distributions within the data, ensuring that the dataset is well-prepared for further analysis. Common EDA techniques include visualizing data distributions, checking for missing values, identifying outliers, and assessing feature relationships using statistical methods.

EDA plays a vital role in detecting issues within the dataset, such as duplicate records, inconsistent data types, or misleading patterns. Tools like Pandas, Matplotlib, Seaborn, and ydata-profiling allow for effective visualization and analysis. Automating EDA can provide quick insights, reducing the manual effort required for initial data exploration. In your project, the automatic generation of EDA reports simplifies this process, making it more accessible for users by instantly revealing data quality metrics and correlations.

Integrating EDA within an AutoML pipeline enhances model selection and feature engineering. By understanding the dataset early, data preprocessing steps can be refined, leading to better predictive accuracy. The challenge lies in ensuring the insights gained from EDA remain interpretable, guiding users toward meaningful decisions rather than overwhelming them with excessive information.

1.7 AUTOMATED MODEL SELECTION

Automated model selection simplifies the machine learning workflow by automating the evaluation and comparison of various algorithms. It eliminates the need for extensive manual experimentation, ensuring efficient identification of the best-performing model based on metrics like accuracy, precision, recall, and F1 score. Hyperparameter tuning, a core component of this process, enhances model performance by optimizing parameters systematically. Feature engineering is also automated, transforming raw data into a format that is more suitable for analysis, thus improving the model's predictive capability.

Frameworks like PyCaret, Auto-sklearn, and H2O.ai exemplify the practical application of automated model selection. These tools streamline data preprocessing, automate model training, and compare algorithms to determine the most optimal solution for the dataset at hand.

Their adaptability extends to diverse data types, such as numerical, textual, and image data, making them robust and versatile in real-world applications. Integration of such systems accelerates workflow while ensuring high-performing and scalable results.

Despite the advantages, challenges remain, such as ensuring interpretability and addressing issues in complex datasets with multimodal inputs like images or audio. Future advancements aim to make automated model selection more

transparent and capable of handling dynamic, real-time environments. By reducing barriers for non-experts, automated model selection contributes to democratizing access to machine learning and scaling predictive analytics across industries.

1.8 ENSEMBLE LEARNING TECHNIQUES

Ensemble learning techniques, such as Random Forest, Gradient Boosting, and Extra Trees, play a pivotal role in automating machine learning workflows. These methods combine multiple algorithms to create robust predictive models that outperform individual classifiers, ensuring high accuracy and reliability. In the context of your project, these ensemble methods are integral to improving the scalability of predictive analytics for survey data. By automating the process of training and comparing these models within a workflow, the project not only achieves optimal performance but also reduces manual intervention, streamlining complex tasks like feature selection and hyperparameter optimization.

Within your automated workflow, Random Forest contributes by building numerous decision trees, each trained on different subsets of the survey dataset. By aggregating predictions from all trees, the algorithm minimizes overfitting and handles class imbalances effectively, making it particularly suitable for varied survey data distributions. Gradient Boosting further enhances predictive accuracy by iteratively learning from errors in previous models, fine-tuning predictions with each step. Meanwhile, Extra Trees introduces additional randomness in feature selection, increasing diversity among decision trees and ensuring robustness in handling survey datasets with varying levels of noise or complexity.

These ensemble techniques thrive in automated workflows by seamlessly integrating model selection and evaluation stages. By leveraging platforms like PyCaret and Auto-sklearn, your project harnesses the power of these algorithms without requiring extensive manual configurations. Automated workflows compare models based on metrics such as accuracy, F1 score, and recall, identifying the best-performing ensemble method for predictive analytics. This end-to-end automation ensures that predictive models adapt dynamically to diverse survey data environments, delivering actionable insights with reliability and scalability.

1.9 CROSS VALIDATION AND EVALUATION

Cross-validation is a method used to evaluate how well a machine learning model performs on unseen data. It involves splitting the dataset into smaller sections, called "folds," and training the model on one section while testing it on another. This process is repeated multiple times to ensure the model works well across different portions of the dataset. Automating cross-validation makes this process faster and easier, ensuring consistent results without manual effort. It helps to identify issues like overfitting, where a model performs well on training data but poorly on new data, and underfitting, where the model struggles to learn patterns altogether.

Evaluation automation goes hand in hand with cross-validation. It uses predefined metrics like accuracy, precision, recall, and F1 score to assess the quality of a model's predictions automatically. This ensures that the model is evaluated thoroughly across various aspects of its performance, such as its ability to make correct predictions and avoid mistakes. Tools and platforms that automate these evaluations can also generate visual reports, such as confusion matrices and performance charts, helping users understand the model's strengths and weaknesses at a glance.

Together, cross-validation and evaluation automation streamline the testing and selection of machine learning models, ensuring that the most reliable and high-performing model is chosen. These automated processes save time, reduce human error, and make machine learning workflows more accessible for users, even those with limited technical expertise. In your project, this helps ensure that predictive analytics on survey data is accurate, scalable, and ready for real-world applications.

1.10 OBJECTIVES

The main objectives of the project are listed below

- To automate and streamline the early stages of ML workflows by simplifying dataset ingestion, standardization, and exploratory data analysis.
- To enhance data quality and understanding through automated EDA reports that highlight key insights, distributions, and anomalies.
- To establish a scalable foundation for full AutoML integration, enabling future automation of model selection, training, and adaptive learning.

1.11 ORGANIZATION OF THESIS

Chapter 1 introduces automated ML workflows, addressing challenges in data preprocessing, EDA, and model selection. It explores AutoML's role in improving efficiency through ingestion, standardization, and analysis.

Chapter 2 reviews existing literature and research related to automating machine learning workflows. It covers techniques like AutoML, meta-learning, and adaptive frameworks, analyzing their contributions and limitations in areas such as model selection, optimization, and handling complex data.

Chapter 3 details the proposed system and its components. It explains the framework for automating machine learning workflows, including data ingestion,

preprocessing, automated EDA, and AutoML-driven model training and selection.

Chapter 4 describes the implementation of the automated machine learning workflow system, including the tools and libraries used. It covers the use of Python libraries like Pandas, NumPy, and scikit-learn for data manipulation, numerical computing, and model training. The chapter also details the datasets used for the project and the implementation of key modules.

Chapter 5 presents the results and analysis of the implemented system, evaluating its performance on the chosen datasets. It includes visualizations and discussions of the system's strengths and weaknesses. The chapter provides insights into the effectiveness of the automated workflow in achieving its objectives.

Chapter 6 summarizes the project's key outcomes and contributions, highlighting its impact on streamlining machine learning workflows. It also discusses potential future research directions and enhancements to the system. The chapter concludes by emphasizing the potential of the work to advance the field of automated machine learning.

CHAPTER 2

LITERATURE SURVEY

2.1 OVERVIEW

The research explores advanced approaches to streamline machine learning workflows, focusing on areas such as AutoML, meta-learning, and adaptive frameworks. These methods address key challenges like data preprocessing, model selection, and scalability in complex environments. Techniques such as dynamic ensemble selection, adaptive graph learning, and online AutoML aim to enhance model efficiency, scalability, and adaptability. By addressing limitations, such as reliance on data quality and computational inefficiencies, the research lays a foundation for robust and generalizable AI systems, improving predictive analytics and data-driven decision-making.

2.2 MULTI-MODEL / ADAPTIVE MODEL SELECTION

T. Hospedales(2025) proposes a Multi-Model Selection System for preprocessing diverse raw input types. It extracts relevant features from text, numbers, images, audio, and video for model selection. The system integrates automated feature extraction, dynamic model adaptation, and real-time inference. It enhances accuracy in various domains like social networks and finance through adaptive learning. However, it has high computational overhead, complexity in feature extraction, and potential biases. These challenges impact efficiency but are crucial for optimizing model performance dynamically.

A. Ghosh et al.(2024) proposes an Adaptive Contextual Bandit (ACB) algorithm for model selection. It eliminates overly simple models in phases while minimizing regret in stochastic contextual bandits. The system achieves a regret

rate matching an optimal algorithm with true model class knowledge. An alternative Explore-Then-Commit (ETC) method is introduced, offering a simpler but costlier approach. Limitations include computational overhead, reliance on realizability, and higher regret in simple models. Handling highly dynamic environments remains a challenge for the proposed system.

Z. Liu et al.(2023) propose an adaptive model selection framework for offline multiobjective optimization. It integrates Kriging models and RBFNs, selecting the best model based on an error threshold. Using the IBEA-MS algorithm, it improves search robustness and reduces misleading rankings. Experiments show effectiveness on benchmark problems with up to 30 decision variables. Limitations include high computational cost, sensitivity to initial data, and handling high dimensions. The system also depends on surrogate model accuracy and struggles in dynamic environments.

N. Shukla et al.(2019) propose an Adaptive Model Selection Framework for airline pricing using Thompson sampling. It dynamically routes customer requests to the best-performing pricing model for optimization. The system balances exploration and exploitation to improve pricing decisions. Revenue per offer increases by 43%, and conversion scores improve by 58%. Limitations include high computational cost, sensitivity to exploration-exploitation balance, and data dependency. Challenges also include market adaptation and limited generalization beyond airline pricing.

Han et al. (2023) proposed a dynamic ensemble selection algorithm that uses a sliding window mechanism to adapt to imbalanced and drifting data streams. The method updates the ensemble composition in real time, based on local accuracy. It improves performance under concept drift scenarios. However, it struggles with abrupt drift and relies on pre-defined window sizes. This limits adaptability in highly dynamic data environments. The model also assumes class imbalance patterns remain somewhat consistent.

2.3 AUTOML AND DRL FOR OPTIMIZATION

F. Ming et al. (2024) propose a DRL-assisted operator selection framework for CMOEAs. It uses a deep Q-learning network (DQN) to select evolutionary operators dynamically. The system adapts based on convergence, diversity, and feasibility for better optimization. Integrated into four CMOEAs, it outperforms nine state-of-the-art methods on 42 benchmarks. D. Basu (2024) introduces an efficient AutoML framework using meta-features and dynamic configuration adaptation for improved model selection. However, its reliance on predefined meta-features may limit adaptability in novel datasets.

Roy (2024) introduced an AutoML system driven by dataset-specific meta-features and adaptive configuration strategies. Their approach allows automated pipeline selection based on the data's characteristics. It reduces trial-and-error and improves runtime efficiency. However, the model is sensitive to inaccurate or insufficient meta-feature extraction. Its generalizability to unseen data types is also limited. Further, adaptation may fail in out-of-distribution settings.

Ma et al. (2024) introduced AutoSelector, a meta-learning feature selection framework for large datasets. It dynamically selects relevant features using meta-knowledge from prior tasks. This boosts scalability and selection speed. However, it assumes meta-features are reliable and consistent. The method may misperform on highly novel data distributions. Interpretability is also limited for non-linear selection paths.

Garcia and Brueckner (2024) proposed a bandit-based AutoML framework using uncertainty-aware reward signals. It dynamically explores configurations with efficient decision-making. The system improves pipeline search in uncertain environments. However, reward estimation can lag and mislead optimization. It requires extensive tuning of reward strategies. Real-time performance may vary across domains.

2.4 META-LEARNING & RAPID ADAPTATION

M. Gutowska et al.(2023) proposes a meta-learning approach for unsupervised anomaly detection by selecting the best algorithm based on dataset-specific meta-features. The system improves upon traditional trial-and-error methods by leveraging past performance data to recommend the most suitable anomaly detection technique. C. Zhang et al. (2024) Proposes an efficient multi-view unsupervised feature selection method using adaptive structure learning. However, the approach may struggle with scalability in extremely high-dimensional data.

A. Vettoruzzo et al.(2024) proposes a meta-learning framework for rapid model adaptation with minimal data. It integrates optimization-based, model- based, and metric-learning approaches for better generalization. Techniques like few-shot learning, self-supervised learning, and domain adaptation enhance efficiency. Z. Shen et al. (2024) Introduces a causally-aware unsupervised feature selection learning method for improved feature relevance. However, the method faces challenges in ensuring accurate causal relationships in noisy data.

Liu et al. (2024) developed a hybrid framework combining meta-learning and AutoML for mixed data types. It supports both structured and unstructured inputs in an integrated system. Automation spans preprocessing and modeling. However, handling heterogeneous data introduces pipeline overhead. Inference latency can become a bottleneck. Complex preprocessing requires manual overrides. Wan et al. (2025) proposed a task-adaptive selection strategy in meta-learning. Their system dynamically matches tasks with effective models based on performance signals. It improves learning across variable distributions. However, task similarity measures may be unreliable. The model may misallocate resources under ambiguous signals. Generalization across domains needs further testing.

2.5 LLM LIFECYCLE & OPERATIONS

S. Pahune(2025) proposes LLMOps (Large Language Model Operations), a specialized framework for managing the lifecycle of large language models (LLMs) such as GPT and BERT. It extends MLOps practices by incorporating tools like Vertex AI to handle fine-tuning, deployment and monitoring. LLMOps addresses challenges such as model drift, scalability, and ethical concerns. By ensuring continuous monitoring and adaptation, LLMOps enhances the performance and reliability of LLM-based applications in dynamic environments. However, LLMOps also faces limitations, including high computational costs due to extensive GPU/TPU requirements.

2.6 INCREMENTAL/CONTINUAL LEARNING

Q. Hu et al.(2022) proposes a curiosity-driven class-incremental learning (CdCIL) framework. It selects samples based on uncertainty and novelty to prevent catastrophic forgetting in deep learning. The system prioritizes curious samples to refine decision boundaries and improve generalization. An adaptive sampler guides learning, reducing inefficient updates while preserving old and new knowledge. P. Sellars et al. (2024) Proposes LaplaceNet, a hybrid graph-energy neural network for deep semisupervised classification. However, its complexity in training may limit its practical deployment in resource-constrained environments.

B. Celik et al.(2023) proposes OAML, an adaptive AutoML framework for online learning. It optimizes pipelines in real-time using asynchronous genetic programming and successive halving. Backup ensembles, model stores, and drift detection ensure fast adaptation to concept drift. OAML outperforms standard online learning methods on real and synthetic data streams. It is ideal for finance, cybersecurity, and IoT analytics. Limitations include high computational costs, scalability issues, and sensitivity to drift detection errors.

Patel and Srinivasan (2024) introduced a continual learning model with uncertainty-aware gradient adaptation. It reduces forgetting by modulating updates based on uncertainty. The system adapts well to sequential tasks. However, uncertainty estimation is sensitive to noise. Gradient modulation can destabilize convergence. The approach needs precise tuning for real-time tasks.

Chen et al. (2024) proposed an online class-incremental learning method with adaptive sample memory replay. It maintains a small memory of past samples to stabilize learning. The method balances stability and plasticity over time. However, the fixed memory size may limit long-term learning. Class overlap and memory diversity are hard to manage. Replay efficiency reduces as task complexity grows.

Wei et al. (2024) proposed a few-shot class-incremental learning method that leverages feature space augmentation and self-supervised learning. This enables fast generalization to new classes with minimal forgetting. The approach is scalable and adaptive to new tasks.

However, performance degrades under large domain shifts. The feature augmentation may amplify irrelevant patterns. It also assumes sufficient initial class separation.

Han et al. (2023) proposed a dynamic ensemble selection algorithm that uses a sliding window mechanism to adapt to imbalanced and drifting data streams. The method updates the ensemble composition in real time, based on local accuracy. It improves performance under concept drift scenarios.

However, it struggles with abrupt drift and relies on pre-defined window sizes. This limits adaptability in highly dynamic data environments. The model also assumes class imbalance patterns remain somewhat consistent.

2.7 ADAPTIVE REPRESENTATION AND PREDICTION

Shaojie Qiao et al.(2021) proposes a self-adaptive trajectory prediction approach using HMMs. The system introduces HMTP for continuous trajectory prediction and HMTP* for dynamic adjustments. A density-based trajectory partitioning algorithm improves clustering and prediction accuracy. It outperforms traditional methods, benefiting intelligent transportation and location-based services. Limitations include high computational cost, sensitivity to parameters, and handling highly dynamic movement. Scalability issues and potential errors in state transitions also affect its performance.

P. Mishra et al.(2024) proposes an automatic title generation system using GPT-2. It follows a three-step pipeline: Generation, Selection, and Refinement for concise titles. GPT-2 generates multiple titles, a heuristic method selects the best, and refinement ensures accuracy. The system improves title relevance even with limited training data, performing well on ROUGE and BLEU. H. Chen et al. (2024) presents an online class-incremental learning method with adaptive sample memory replay for continual learning. However, its reliance on memory may lead to scalability issues in large-scale environments.

P. Huang et al.(2024) proposes an unsupervised feature selection framework using CAG-U and CAG-I. It dynamically learns an adaptive graph to preserve data structure while selecting uncorrelated features. The models prevent excessive structural deviations by controlling differences between initial and final graphs. Discrete projection matrices enhance efficiency in high-dimensional data processing.

R. Wang et al. (2023) introduces a graph-based meta-learning approach for unsupervised feature selection. However, it may not scale well to extremely large graph datasets due to computational overhead.

2.8 DATA STREAM PROCESSING

B. Jiao et al(2024) proposes the DES-ICD framework for imbalanced data streams with concept drift. It selects classifiers based on local performance rather than global accuracy for better adaptability. AnnSMOTE, an adaptive oversampling method, generates new minority instances from data distribution changes. M. Han et al. (2023) Proposes a dynamic ensemble selection classification algorithm for imbalanced drift data streams using window-based methods. However, it suffers from computational complexity and requires continuous adaptation to evolving data streams.

M. Böther et al(2025) proposes Modyn, a data-centric ML pipeline orchestrator for growing datasets. It enables users to define data selection and retraining policies for handling distribution shifts. Modyn optimizes sample-level data selection, reducing training costs while maintaining accuracy. Composite models ensure fair performance comparisons across different training strategies. N. Patel and H. Srinivasan (2024) introduces continual learning via uncertainty-aware gradient-based adaptation for model selection. However, the method may be prone to overfitting in highly variable data streams.

F. Ming(2024) introduces a novel Deep Reinforcement Learning-assisted framework to optimize operator selection in constrained multi-objective optimization problems (CMOPs). It uses population metrics like convergence, diversity, and feasibility as states, evolutionary operators as actions, and the improvement in population state as rewards.

By employing a deep Q-network, it adaptively selects the best operator for the problem at hand, significantly enhancing algorithmic performance across diverse optimization scenarios. The framework is versatile and can integrate any number of operators into existing CMOP algorithms but the system may underperform in CMOPs that inherently favor specific operators that are distributed across all operators during the learning stage.

Table 2.1 – Summary on AUTOML Approaches

SI. NO	Topic	Author	Proposed Work	Limitations
1.	Transitioning from MLOps to LLMOps: Navigating the Unique Challenges of Large Language Models	S. Pahune, Z. Akhtar (2025)	This paper compares LLMOps (Large Language Model Operations) with MLOps, focusing on the unique challenges, infrastructure needs, and methodologies for managing LLM lifecycles,such as fine-tuning, deployment,and monitoring.	Complex scalability requirements for LLMs in production environments.
2.	Meta- Learning in Neural Networks: A Survey	T.Hospedales, A.Antoniou, P. Micaelli, A. Storkey (2022)	A comprehensive survey on meta-learning,introducing a new taxonomy to categorize current methods and exploring its applications in few shot learning and reinforcement learning.	Accurately categorizing diverse meta learning methods and struggles with complex multimodal task distributions.
3.	Constructing a Meta-Learner for Unsupervised	M. Gutowska, S. Little, A. Mccarren	A meta-learning approach selecting the best anomaly detection	Limited generaliation across all anomaly

	Anomaly Detection	(2023)	algorithm based on meta features from unlabeled data, outperforming existing methods.	detection tasks and reliance on high quality meta features for accurate selection.
4.	Model Selection for Generic Contextual Bandits	A. Ghosh, A. Sankaraman, K. Ramchandan (2024)	An adaptive algorithm selects models for contextual bandits, minimizing selection costs while offering sharper guarantees in linear contexts.	Higher model selection cost in ETC compared to ACB.
5.	Performance Indicator-Based Adaptive Model Selection for Offline Data- Driven Multi objective Evolutionary Optimization	Z. Liu, H. Wang, Y. Jin (2023)	An adaptive surrogate-assisted evolutionary algorithm used for switching between Kriging models and RBFNs based on approximation error.	Depends only on surrogate model quality.
6.	Curiosity- Driven Class- Incremental Learning via Adaptive	Q. Hu, Y. Gao, B. Cao (2024)	A curiosity-driven class incremental learning approach is proposed to reduce catastrophic forgetting by	Dependence on accurate quantification of uncertainty and novelty.

	Sample Selection		selectively learning informative samples based on uncertainty and novelty, improving model robustness and balance between new and old knowledge.	
7.	A Self- Adaptive Parameter Selection Trajectory Prediction Approach via Hidden Markov Models	S. Qiao, D. Shen, X. Wang, N. Han (2024)	A hidden Markov model based trajectory prediction algorithm (HMTP) is proposed to improve accuracy in network- constrained environments. An enhanced version, HMTP*, introduces self-adaptive parameter selection and a density-based partitioning algorithm for better prediction efficiency.	Potential inefficiency in highly dynamic environments with abrupt speed variations.
8.	Dynamic Ensemble Selection for Imbalanced Data Streams with Concept	B. Jiao, Y. Guo (2022)	A dynamic ensemble selection method (DES-ICD) is proposed for handling concept drift and class imbalance in data streams. It employs AnnSMOTE	Increased computation al complexity due to local classifier selection.

	Drift		to balance data and selects base classifiers dynamically based on local performance rather than global accuracy.	
9.	Automatic Title Generation for Text with Pretrained Transformer Language Model	P. Mishra, C. Diwan,S. Srinivasa, G. Srinivasaraghavan (2019)	A model that adapts and generalizes using meta-learning for automatic title generation from text.	Handling complex, multimodal task distributions.
10.	Unsupervised Feature Selection via Controllable Adaptive Graph Learning and Discriminative Feature Learning	P. Huang, M. Xie, X. Yang (2024)	Unsupervised feature selection faces Challenges in preserving intrinsic structure and finding uncorrelated features. CAG-U and CAG-I use adaptive graph learning and discrete projection to overcome inefficiencies in high-dimensional data.	Inefficiency in handling extremely highdimensional datasets.
11.	Modyn: DataCentric Machine Learning	M. Bother, T.Robroek,	Modyn, a data-centric ML platform, enables continuous model training on	Effectiveness of data selection policies depends

	Pipeline Orchestration	V.Gsteiger, R. Holzinger, X. Ma, P. Tozun (2025)	growing datasets by applying data selection and triggering policies to optimize retraining costs.	heavily on the quality and representativeness of the data.
12.	Advances and Challenges in Meta-Learning: A Technical Review	A. Vettoruzzo, M.R. Bouguelia, J. Vanschoren (2024)	A meta-learning system to enhance model adaptation and generalization across diverse tasks, particular in data scarce scenarios. It integrates multi-task learning, transfer learning and continual learning to improve efficiency and performance.	Adaptation to data distribution shifts in dynamic environments.
13.	Online AutoML: An Adaptive AutoML Framework for Online Learning	B. Celik, P. Singh, J. Vanschoren (2023)	An adaptive Online Automated Machine Learning (OAML) system that continuously optimizes learning pipelines in dynamic environments with evolving data distributions.	Inefficiencies in handling highly complex or largescale data stream.
14.	Adaptive Model Selection Framework: An Application to	N. Shukla, A. Kolbeinsson, L. Marla,	An adaptive meta- decision framework using Thompson sampling to dynamically route	Assumes a consistent data stream and prior knowledge.

	Airline Pricing	K. Yellepeddi (2019)	customer requests to different airline pricing models.	
15.	Multitask Learning with Meta- Learning: Optimizing Learning of Multiple Related Tasks	A. Ruder, M. E. Peters, M. L. M. Asano (2024)	A multitask learning framework optimized through meta learning, enhancing generalization across a wide array of tasks and datasets.	High dependence on prior data quality and representativeness for successful multitask learning.
16.	Constrained Multi-Objective Optimization With Deep Reinforcement Learning Assisted Operator Selection	F. Ming, W. Gong, L. Wang and Y. Jin (2024)	The system may underperform in CMOPs that inherently favor specific operators, as computational resources are distributed across all operators during the learning stage.	The system may underperform in CMOPs that inherently favor specific operators.

2.9 SUMMARY

The studies discussed in the research provide a comprehensive perspective on various methodologies aimed at optimizing machine learning models across different domains. By exploring LLMOps for managing large-scale language models, meta-learning for adaptive model selection, and AutoML for automating ML workflows, these approaches significantly enhance efficiency, scalability, and decision-making. Each methodology plays a crucial role in overcoming common challenges associated with ML deployment, such as hyperparameter tuning, model selection, and real-time adaptability.

Despite these advancements, several limitations persist, including high computational costs, sensitivity to hyperparameter selection, and ethical concerns related to automated learning systems. Addressing these challenges requires continued research into making machine learning systems more transparent, interpretable, and adaptable to diverse data environments. The evolution of these techniques paves the way for more robust AI models capable of handling complex, multimodal datasets while reducing the burden on human expertise. Ultimately, the insights gained from these studies contribute to the development of scalable, efficient, and responsible AI-driven decision-making systems that can be applied across industries.

CHAPTER 3

PROPOSED WORK

(AUTOML-DRIVEN INSIGHTS FROM SURVEY DATA)

3.1 INTRODUCTION

The proposed system aims to simplify the machine learning process for users, particularly those without advanced expertise in the field. Built using Python, Scikit-learn, and Streamlit, the system automates critical tasks such as data preprocessing, model selection, and performance evaluation, enabling users to upload datasets, select target variables, and automatically process and split the data into training and testing sets. The system supports several popular machine learning models, including Random Forest, Gradient Boosting, and Support Vector Machines (SVM), which are evaluated using cross-validation and performance metrics like accuracy and F1 score.

The best-performing model is selected and retrained on the entire dataset to ensure optimal results. With an intuitive interface powered by Streamlit, the system provides users with easy access to machine learning workflows, allowing them to quickly view performance scores and compare models. This AutoML system is designed to democratize machine learning by automating complex processes, making it accessible to users with minimal technical knowledge.

3.2 PROPOSED WORK

The proposed work explains the design and development of an Automated Machine Learning (AutoML) system that simplifies the machine learning process for users. Built using Python, Scikit-learn, and Streamlit, the system

automates key tasks like data preparation, model selection, and performance evaluation, making it easier for users without advanced machine learning knowledge to build and evaluate models. Users can upload their datasets, choose the target variable (what they want to predict), and the system automatically processes the data and splits it into training and testing sets.

The system supports several common machine learning models, such as Random Forest, Gradient Boosting, and Support Vector Machines (SVM). These models are trained using cross-validation, and performance metrics like accuracy and F1 score are calculated to compare how well each model performs. After testing all models, the best one is selected and retrained using the entire dataset to give the best possible results.

Streamlit provides an easy-to-use web interface where users can upload their data, select the target variable, and start training models with just a few clicks. The results, including performance scores, are displayed on the dashboard, allowing users to quickly understand which model works best for their data.

The goal of this system is to automate complex machine learning tasks so that anyone can use machine learning without needing to be an expert. The system could also allow users to export trained models for use in real-world applications.

Figure 3.1 illustrates the block diagram of the proposed system. It begins with dataset upload and validation, followed by cleaning and standardization of data, including handling missing values. An automated exploratory data analysis (EDA) report is generated to provide insights into distributions and correlations. This module lays the groundwork for seamless integration into the machine learning workflow.

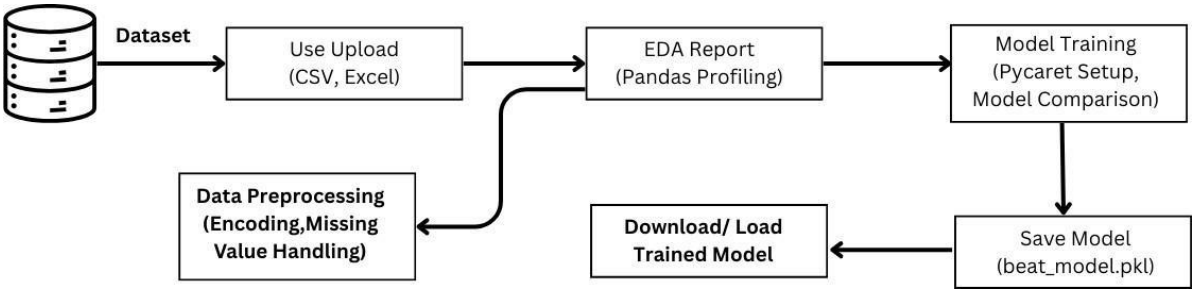


Figure 3.1 Block Diagram of AutoML-Driven Insights from Survey Data

3.3 WORKING PROCESS OF THE PROPOSED WORK

From the insights of figure 3.1, the proposed work is split into 3 main modules which are,

- **Data ingestion and Data preprocessing**

  This module is responsible for accepting and preparing datasets for analysis. Users can upload files in commonly used formats such as CSV, which are then parsed and standardized to ensure compatibility with downstream processes. The system performs initial preprocessing steps including cleaning column names, detecting data types, handling missing values in categorical and numerical columns through imputation, and encoding categorical variables. An Exploratory Data Analysis (EDA) report is generated to give insights into data distributions, correlation patterns, and outliers. This not only assists users in understanding the dataset but also ensures that the data is in optimal condition for training machine learning models.

- **Model Training and Selection**

  Following data preparation, the system proceeds with model training and selection, where categorical features are label-encoded, and feature scaling is applied as needed to enhance model performance. The AutoML engine, utilizing Scikit-learn and potentially integrated with frameworks like PyCaret, automates the process of model comparison. A set of predefined classifiers,

including Random Forest, Gradient Boosting, LogisticRegression, and others, are evaluated using LightGBM. Performance metrics such as accuracy and F1-score are computed to identify the best-performing model. Once the top model is selected, it is retrained on the full dataset and saved for future predictions, ensuring optimal generalization and performance.

- **Model Comparison**

  This module evaluates multiple machine learning models by training them on preprocessed data, which has undergone cleaning, encoding, and scaling to ensure compatibility with various algorithms. Models such as Random Forest, Logistic Regression, AdaBoost, and Gradient Boosting are tuned using GridSearchCV to optimize performance based on cross-validation scores. The best model is selected based on its metrics, like accuracy, and is further validated on unseen test data to ensure robust real-world performance.

3.3.1 DATA INGESTION AND DATA PREPROCESSING

The Data Ingestion and Preprocessing module is a crucial component of the AutoML system, responsible for accepting raw input data and transforming it into a clean, consistent, and analysis-ready format. This module simplifies the typical preprocessing workflow by automating essential tasks such as file validation, column cleaning, missing value handling, encoding, and data visualization.

It ensures that the uploaded dataset is of high quality and structurally ready for training machine learning models. Users interact through a simple web interface powered by Streamlit, where they can upload CSV files and trigger the preprocessing pipeline.

Once a file is uploaded, the system initiates a series of preprocessing steps. These include validating the file format, cleaning column names, filling in

missing values, and encoding categorical data into numerical form. An optional but highly valuable feature of this module is the automatic generation of an Exploratory Data Analysis (EDA) report. This report provides insights into data distributions, variable correlations, missing data, and potential anomalies, and is delivered in an interactive HTML format for easy understanding. Overall, this module reduces the technical barrier for users by automating data cleaning and analysis, enabling them to focus on interpreting results and making decisions.

Steps:

1. Upload Dataset:Users upload the dataset using Streamlit's file uploader component in formats like CSV or Excel.
2. File Type Check:The system validates the uploaded file format to ensure it is readable and compatible with the pipeline.
3. Save Dataset Locally:The validated dataset is saved as source_data.csv for persistent access during further processing.
4. Standardize Column Names:Column headers are cleaned and standardized (e.g., removing special characters, converting to lowercase, replacing spaces with underscores) to maintain consistency.
5. Detect Data Types:The system automatically infers the data types of each column (numerical, categorical, text, etc.) to apply suitable preprocessing methods.
6. Handle Missing Values:Missing values are addressed,

    - Categorical columns: filled with the mode or a placeholder like "Unknown".
    - Numerical columns: imputed using the mean or median.

7. Encode Categorical Features:Label Encoding or One-Hot Encoding is applied to categorical columns to convert them into numerical format compatible with machine learning models.

8. Generate EDA Report:An automated Exploratory Data Analysis (EDA) report is generated using libraries like pandas_profiling or ydata-profiling, offering statistical summaries, distributions, and correlations.
9. Generate Interactive HTML Report:The EDA output is saved as an interactive HTML file and rendered in the Streamlit app for user-friendly data exploration.

Figure 3.2 illustrates the working flow of the data ingestion and data preprocessing module. It streamlines the initial stages of data processing, ensuring seamless preparation for further analysis and model training. This workflow ensures efficiency, scalability, and automation in predictive analytics. By automating tasks like validation, missing value handling, and standardization, it minimizes manual intervention. Additionally, the automated exploratory data analysis (EDA) provides insightful visualizations, enabling quicker and more informed decisions.

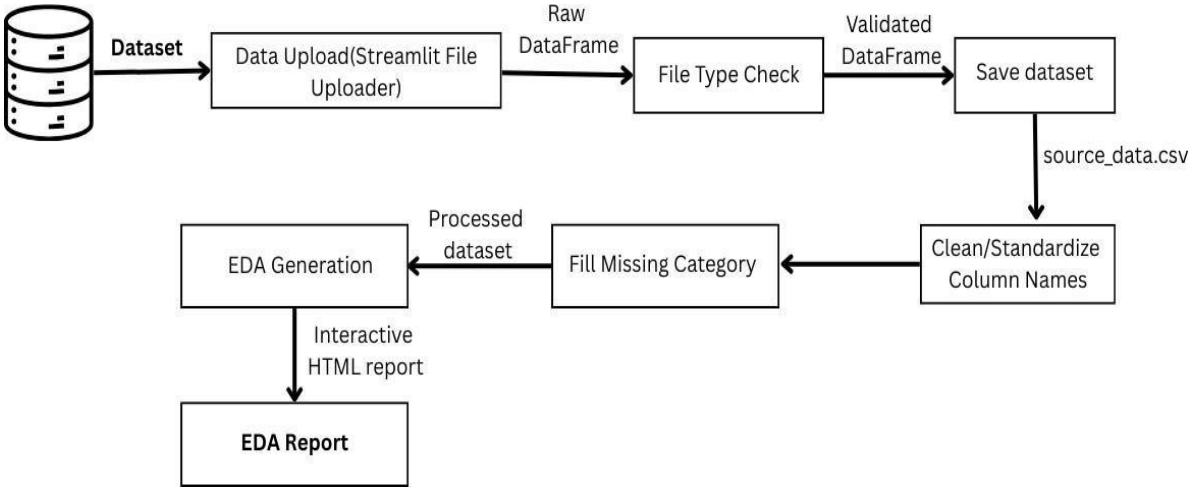


Figure 3.2 Work flow of Data ingestion and data preprocessing module

3.3.2 Pseudo Code for Data ingestion and data preprocessing Module

Input: User-uploaded dataset file (.csv or .xlsx) via Streamlit File Uploader.

Output: Cleaned and saved dataset (source data.csv) and automatically generated EDA Report.

```
procedure UPLOAD_AND_EDA(file_uploader, profiler)
    file ← file_uploader.receive_file()
    if file is None then
        return "No file uploaded"
    end if
    if file.type == 'csv' then
        df ← ReadCSV(file)
    else if file.type == 'xlsx' then
        df ← ReadExcel(file)
    else
        return "Unsupported file format"
    end if
    SaveCSV(df, "source_data.csv")
    CleanedDF ← df.copy()
    for each column in CleanedDF.columns do
        if column name is None or empty then
            Replace column name with default name
        end if
    end for
    for each column in CleanedDF do
        if column.type is categorical then
            Replace nulls with "Unknown"
        end if
    end for
    eda_report ← profiler.generate(CleanedDF)
```

```
26:    DisplayProfileReport(eda_report)
27:    return "EDA report generated successfully"
28: end procedure
```

3.3.3 MODEL TRAINING AND SELECTION

After the preprocessing stage, the system seamlessly transitions to the model training and evaluation phase. Here, a range of machine learning algorithms are applied to the cleaned data, and their performance is compared using predefined metrics. To ensure model compatibility, preprocessing steps such as label encoding for categorical features, mode imputation for missing values, and feature scaling for numerical features are performed. Leveraging automation tools like PyCaret, the system efficiently handles the training and testing of multiple models—including Random Forest, CatBoost, and others while utilizing LightGBM not as a training model but as a benchmark evaluator to assist in selecting the best-performing model based on its comparative scoring.

LightGBM plays a critical role in guiding model selection by evaluating performance metrics such as accuracy, F1-score, and AUC across all trained models. Rather than being trained itself, LightGBM is employed to rank and score other candidate models, identifying the one that achieves optimal balance across key metrics. The top-ranked model is then finalized for deployment. Along with this, the system automatically generates comprehensive performance reports for each evaluated model, detailing their respective metrics and selection rationale. This structured, automated approach simplifies complex ML workflows while ensuring adaptability, scalability, and precision in predictive analytics.

Steps:

1. Label Encoding for Categorical Features: Categorical variables are converted into numerical form using label encoding, ensuring compatibility

with machine learning algorithms.

2. Impute Missing Values: Numerical features are imputed using the mean, while categorical ones use the mode, maintaining consistency and avoiding data loss.

3. Feature Scaling: Numerical features are normalized using StandardScaler or MinMaxScaler to align feature scales and enhance model convergence and accuracy.

4. Model Comparison Setup: A set of models such as RandomForest, CatBoost, and others are prepared for training and evaluation; LightGBM is set as a benchmark evaluator.

5. Model Training: Each candidate model is trained using k-fold cross-validation to ensure robust evaluation and reduce the chances of overfitting.

6. Model Evaluation: Trained models are evaluated based on performance metrics like Accuracy, F1-score, and AUC to measure predictive quality.

7. Model Ranking: LightGBM is used to rank models by comparing their evaluation metrics, offering a systematic and objective selection mechanism.

8. Best Model Selection: The top-ranked model, as identified through LightGBM-based comparison, is retrained on the full dataset for final deployment.

Figure 3.3 showcases the model training and selection module, which plays a vital role in ensuring efficient data processing and transformation into formats suitable for machine learning. By automating tasks such as feature encoding, missing value imputation, and scaling, the module facilitates precise model training and evaluation. Leveraging advanced algorithms, it compares multiple models to identify the best-performing one based on defined metrics.

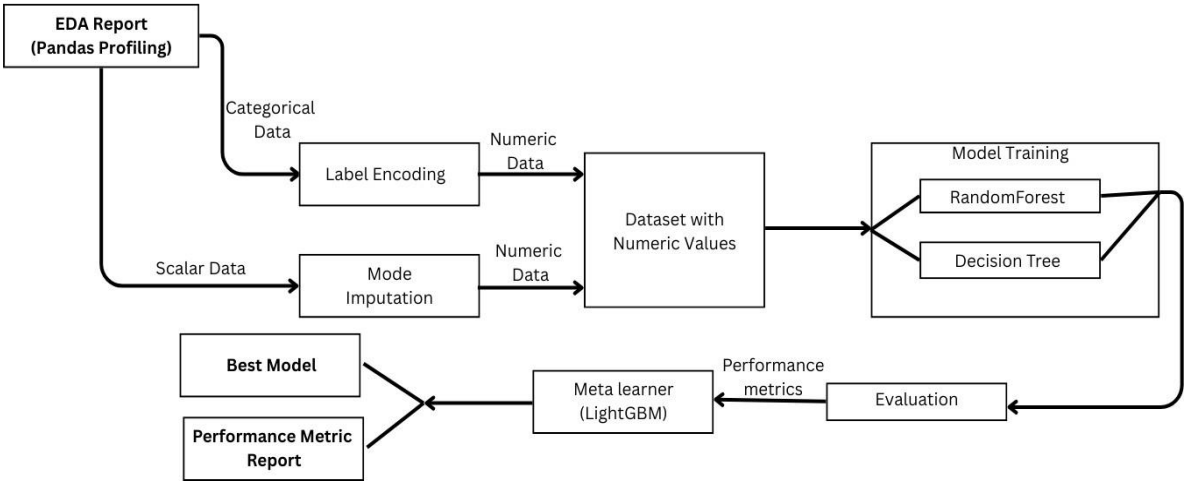


Figure 3.3 Work flow of Model Training and Selection module

3.3.4 Pseudo Code for Model Training and Selection Module

Input: Processed dataset from EDA stage (numeric, labeled data) and user-selected target variable.

Output: Best model saved for deployment or reuse and performance metric report summarizing model evaluation results (Accuracy, AUC, F1-score, etc.).

1. procedure TRAINANDEVALUATE(models, param_grid_xgb, param_grid_rf)
2. data ← LOADDATASET()
3. X_train, X_test, y_train, y_test ← TRAINTESTSPLIT(data.features, data.labels)
4. scaler ← StandardScaler()
5. X_train_scaled ← SCALING(scaler.fit_transform(X_train))
6. X_test_scaled ← SCALING(scaler.transform(X_test))
7. best_model ← INITIALIZE(None)
8. model_performance ← INITIALIZE(Empty_Dictionary)
9. xgb_tuned ← GRIDSEARCHCV(param_grid_xgb, X_train_scaled, y_train)
10. rf_tuned ← GRIDSEARCHCV(param_grid_rf, X_train_scaled, y_train)
11. FOR EACH model_name, model IN models:
12. IF model_name IS 'XGBoost': model ← xgb_tuned.best_estimator

13. IF model_name IS 'Random Forest': model ← rf_tuned.best_estimator_
14. model.fit(X_train_scaled, y_train)
15. accuracy, f1, auc ← CALCULATEMETRICS(model, X_test_scaled, y_test)
16. model_performance[model_name] ← {'accuracy': accuracy, 'f1_score': f1, 'auc': auc}
17. END FOR
18. best_model_name ← GETBESTMODEL(model_performance)
19. best_model ← RETRIEVEMODEL(models, best_model_name)
20. best_model.fit(data.features, data.labels)
21. SAVEFILE(joblib.dump(best_model, "best_model.pkl"))
22. cv_scores ← CROSSVALIDATION(LogisticRegression(), X_train_scaled, y_train, cv=5)
23. DISPLAY(best_model_name, model_performance[best_model_name], cv_scores.mean())
24. end procedure

3.3.5 MODEL COMPARISON

The Model Comparison module serves as a vital component for evaluating multiple machine learning models using processed data that has undergone cleaning, encoding, and scaling. By ensuring compatibility across a variety of algorithms, the module facilitates systematic training and testing of models such as Random Forest, Logistic Regression, AdaBoost, and Gradient Boosting. Hyperparameter tuning, carried out using tools like GridSearchCV, ensures that the models are optimized to achieve their best possible performance. Metrics such as accuracy, F1-score, and cross-validation scores are computed for each model to provide a comprehensive comparison.

Once the models are evaluated, the system identifies the highest-performing model based on the predefined metrics. The selected model is further validated on unseen test data to ensure its reliability and robustness in real-world

applications. This final validation step guarantees that the model is capable of handling practical tasks effectively.

Steps:

1. Data Preprocessing: The dataset is cleaned, categorical features are encoded, and numerical data is scaled to ensure compatibility across all machine learning algorithms.

2. Model Initialization: Various machine learning models, such as Random Forest, Logistic Regression, AdaBoost, and Gradient Boosting, are initialized for evaluation.

3. Hyperparameter Tuning: GridSearchCV is utilized to fine-tune hyperparameters for the models, ensuring optimal performance based on cross-validation scores.

4. Model Training: Each model is trained using the preprocessed dataset, enabling them to learn patterns from the data.

5. Performance Metrics Evaluation: Metrics such as accuracy, F1-score, and cross-validation scores are computed for each model to assess their effectiveness.

6. Best Model Selection: The model with the highest performance metrics is identified and selected as the best-performing model.

7. Model Validation: The chosen model is tested on unseen data to validate its ability to generalize to real-world scenarios.

3.3.6 Pseudo Code for Model Comparison Module

Input: Trained best model (best model.pkl) amd new unseen dataset for inference.

Output: Best model chosen for deployment.

```
1: procedure MODELSELECTIONWITHS3(bucket_name, object_key)
2:     s3_client ← CREATEAWSCLIENT('s3')
3:     s3_object ← s3_client.get_object(Bucket=bucket_name, Key=object_key)
4:     df ← READCSV(s3_object['Body'])
5:     X ← df.iloc[:, :-1]
6:     y ← df.iloc[:, -1]
7:     X_train, X_test, y_train, y_test ← SPLITDATASET(X, y, test_size=0.2,
random_state=42)
8:     scaler ← StandardScaler()
9:     X_train_scaled ← scaler.fit_transform(X_train)
10:    X_test_scaled ← scaler.transform(X_test)
11:    Evaluate models
12:    model_performance ← {}
13:    for each model_name, model in models do
14:        model.fit(X_train_scaled, y_train)
15:        y_pred ← model.predict(X_test_scaled)
16:        accuracy ← accuracy_score(y_test, y_pred)
17:        f1 ← f1_score(y_test, y_pred, average='weighted')
18:        auc ← roc_auc_score(y_test, model.predict_proba(X_test_scaled),
multi_class='ovr')
19:        model_performance[model_name] ← {accuracy, f1, auc}
20:    end for
21:    best_model_name ← argmax(model_performance, key='accuracy')
22:    best_model_performance ← model_performance[best_model_name]
23:    best_model ← GetModelByName(models, best_model_name)
```

```
    best_model.fit(X, y)
    model_filename ← best_model_name.replace(' ', '_') + '_final_model.pkl'
    joblib.dump(best_model, model_filename)
end procedure
```

3.4 SUMMARY

Chapter 3 provides an in-depth explanation of the proposed machine learning workflow, dividing it into three key modules: Data Ingestion and Preprocessing, Model Training and Selection, and Model Comparison and Prediction. The Data Ingestion and Preprocessing module focuses on standardizing datasets by automating tasks like file validation, missing value handling, and generating exploratory data analysis (EDA) reports, ensuring that datasets are clean and ready for training. The Model Training and Selection module automates key preprocessing operations such as encoding categorical variables, feature scaling, and typecasting. It utilizes tools like PyCaret to train and evaluate multiple machine learning models, selecting the best one based on performance metrics like accuracy and F1-score. The final module, Model Comparison and Prediction, enables seamless integration of the best-trained model into real-world applications by saving, loading, and deploying it for making predictions on new datasets. By streamlining each phase of the machine learning process, the framework ensures scalability, efficiency, and adaptability in predictive analytics.

CHAPTER 4

IMPLEMENTATION

4.1 TOOLS REQUIRED

This chapter provides insight into various tools and libraries used, implementation details along with visualization.

4.1.1 pandas

Pandas is a powerful Python library extensively used for data manipulation and analysis. In this project, pandas enable efficient data handling by allowing users to load datasets in formats like CSV and Excel, clean missing or inconsistent data, and preprocess it for machine learning.

Its DataFrame structure provides a tabular way to organize data, making operations like filtering, grouping, and transforming seamless. For instance, pandas helps with encoding categorical variables and splitting data into features (X) and target variables (y), which are essential steps in building machine learning models.

Additionally, pandas simplifies exploratory data analysis (EDA) by offering built-in statistical functions, such as summarizing datasets and detecting trends or patterns. In this project, pandas is used extensively during the ingestion phase to inspect dataset characteristics like missing values, duplicate entries, or outlier detection. Its integration with visualization libraries like matplotlib and seaborn further enhances understanding of the data before feeding it into machine learning workflows.

4.1.2 os

The os library is a standard Python module that provides functionalities for interacting with the operating system. In this project, it plays a crucial role in file management, such as checking the existence of files, handling file paths, and automating the saving and loading of datasets or models. For example, the os.path.exists() function ensures that required files like source_data.csv or model outputs are available, preventing runtime errors during pipeline execution.

Another key use of the os library in this project is managing dynamic workflows where files are generated and stored at runtime. Whether it's saving the EDA report or organizing model outputs, os ensures compatibility across different operating systems, making the system robust and portable. Its seamless integration enhances the automation capabilities of the machine learning pipeline.

4.1.3 numpy

NumPy is a foundational library for numerical computing, playing a pivotal role in mathematical operations. In this project, it assists with numerical operations like matrix manipulations, calculations of statistical metrics, and preparing datasets for machine learning models.

Its integration ensures that operations like scaling features or computing custom performance metrics (e.g., RMSLE or R^2) are executed efficiently, especially on large datasets.

The library's ability to handle arrays and numerical computations at high speed is invaluable in machine learning workflows. For instance, NumPy is used for scaling and standardizing data before training models, which enhances their accuracy and performance. Additionally, it serves as the backbone for other libraries, such as pandas and scikit-learn, offering consistent and high-performance numerical computing.

4.1.4 joblib

Joblib is a lightweight library designed for saving and loading trained machine learning models efficiently. In this project, it is used to store the best-trained model as a serialized object (.pkl file), which can later be loaded for predictions without retraining. This ensures the reusability of models and simplifies deployment in real-world applications.

Beyond saving models, Joblib also supports caching intermediate results in workflows, improving performance for computationally expensive tasks. For example, if a specific feature transformation or EDA process takes significant time, Joblib can cache the results for reuse. This capability reduces redundant computations and enhances the efficiency of the overall machine learning pipeline.

4.1.5. ydata_profiling (formerly pandas_profiling)

This library automates exploratory data analysis (EDA), making it a key component in this project for generating interactive HTML reports. Using ydata_profiling, the project creates comprehensive summaries of datasets, providing insights like missing values, feature distributions, correlations, and outlier detection. This speeds up the process of understanding data quality and readiness for modeling. The generated reports are not only detailed but also visually intuitive, making them accessible to non-technical users. By replacing manual EDA efforts, ydata_profiling contributes significantly to the efficiency of the pipeline, ensuring that data preprocessing decisions are based on clear and actionable insights.

4.1.6. scikit-learn (sklearn)

Scikit-learn is the cornerstone library for machine learning in this project, providing tools for preprocessing, model training, evaluation, and feature

engineering. It supports tasks like encoding categorical variables, scaling numerical features, and splitting data into training and testing sets. Models such as Random Forest, Gradient Boosting, and SVM are all implemented through scikit-learn, allowing seamless integration of a wide array of algorithms.

Additionally, scikit-learn provides metrics for evaluating both classification and regression models, such as accuracy, F1-score, R^2, and RMSE. Its pipeline functionality ensures a smooth flow from data preprocessing to model evaluation, maintaining reproducibility and reducing code complexity. Scikit-learn's versatility and ease of use make it indispensable for the success of this machine learning project.

4.1.7. pycaret

PyCaret simplifies and automates machine learning workflows, significantly reducing development time. In this project, it is employed for tasks like AutoML (automated model selection) to compare multiple models and identify the best-performing one based on metrics such as accuracy or R^2. Its user-friendly interface abstracts many low-level coding requirements, making machine learning accessible even to non-experts. Furthermore, PyCaret integrates preprocessing, model selection, hyperparameter tuning, and deployment into a single pipeline. By generating detailed reports on model performance, it empowers users to make informed decisions while ensuring the scalability of the system for future data and use cases.

4.1.8. matplotlib

Matplotlib is a versatile library for creating static, publication-quality visualizations. In this project, it is employed for generating visual representations of data distributions, feature relationships, and model performance metrics. This enhances interpretability by offering a clear and concise way to understand complex datasets or results.

The library’s customization options allow for creating tailored visualizations, such as scatterplots, bar charts, and line graphs, to highlight key aspects of data or model behavior. Combined with other libraries like seaborn, matplotlib becomes an essential tool for comprehensive data visualization.

Matplotlib also offers functionality to customize the aesthetics and design of plots, making them suitable for presentations or publications. It supports features like annotations, color palettes, and axis scaling, enabling the creation of highly informative and visually appealing visualizations. Furthermore, its ability to integrate with other Python tools ensures seamless workflow enhancements, allowing data scientists to not only analyze but also effectively communicate results to stakeholders.

4.1.9. seaborn

Seaborn builds on matplotlib, offering advanced statistical visualizations with aesthetically pleasing designs. In this project, it is primarily used to create heatmaps, pair plots, and distribution plots, helping to uncover patterns, correlations, and outliers within the dataset. Seaborn’s integration with pandas DataFrames makes it convenient for working directly with structured data.

Its ability to handle complex visualizations effortlessly makes it ideal for exploring relationships between features and identifying trends. Whether it’s understanding feature importance or analyzing residuals in regression tasks, Seaborn adds depth to the analytical process, making it an indispensable tool in the project’s workflow.

Seaborn builds on matplotlib, offering advanced statistical visualizations with aesthetically pleasing designs. In this project, it is primarily used to create heatmaps, pair plots, and distribution plots, helping to uncover patterns, correlations, and outliers within the dataset. Seaborn’s integration with pandas DataFrames makes it convenient for working directly with structured data.

LIBRARIES USED			
alembic	click	funcy	zipp
altair	cloudpickle	future	wincertstore
anyio	colorama	gensim	wordcloud
argon2-cffi	colorlover	gitdb	yellowbrick
argon2-cffi-bindings	commonmark	GitPython	widgetsnbextension
attrs	cufflinks	greenlet	ydata-profiling
Babel	cycler	gym-notices	statsmodels
backcall	cymem	htmlmin	streamlit
backports.zoneinfo	Cython	idna	streamlit-pandas-profiling
beautifulsoup4	databricks-cli	ImageHash	tabulate
bleach	debugpy	imbalanced-learn	tangled-up-in-unicode
blinker	decorator	importlib-metadata	tenacity
blis	defusedxml	importlib-resources	tensorflow-estimator
PyCaret	docker	ipykernel	terminado
Pandas-Profiling	entrypoints	ipython	Flask
BeautifulSoup	fastjsonschema	ipython-genutils	seaborn

Table 4.1 List of Libraries used

4.2 DATASET DESCRIPTION

The dataset used is likely encompasses a mix of numerical, categorical, or textual information, captured through surveys, observations, or sensors. The dataset enables operations like preprocessing, cleaning, exploratory analysis, model training, and prediction.

4.2.1 TEXT DATASET DESCRIPTION

The tabular dataset is obtained from Kaggle and consists of 250 records of Amazon sales transactions (2025), including details about the products sold, customers, payment methods, and order status. It includes 11 parameters such as

Features	Description
Order ID	Unique identifier for each order (e.g., ORD0001).
Date	Date of the order.
Product	Name of the product purchased.
Category	Product category (Electronics, Clothing, Home Appliances, etc.).
Price	Price of a single unit of the product.
Quantity	Number of units purchased in the order.
Total Sales	Total revenue from the order (Price × Quantity).
Customer Name	Name of the customer.
Customer Location	City where the customer is based.
Payment Method	Mode of payment (Credit Card, Debit Card, PayPal, etc.).
Status	Order status (Completed, Pending, or Cancelled).

Table 4.2 Features and Description of Text Dataset

	A	B	C	D	E	F	G	H
1	Order ID	Date	Product	Category	Price	Quantity	Total Sales	Customer Na
2	ORD0001	14-03-2025	Running Sho	Footwear	60	3	180	Emma Clark
3	ORD0002	20-03-2025	Headphones	Electronics	100	4	400	Emily Johnso
4	ORD0003	15-02-2025	Running Sho	Footwear	60	2	120	John Doe
5	ORD0004	19-02-2025	Running Sho	Footwear	60	3	180	Olivia Wilson
6	ORD0005	10-03-2025	Smartwatch	Electronics	150	3	450	Emma Clark
7	ORD0006	14-03-2025	T-Shirt	Clothing	20	1	20	John Doe
8	ORD0007	18-03-2025	Smartwatch	Electronics	150	4	600	Emma Clark
9	ORD0008	02-03-2025	Smartphone	Electronics	500	1	500	Sophia Miller
10	ORD0009	08-03-2025	T-Shirt	Clothing	20	3	60	Sophia Miller
11	ORD0010	12-03-2025	Smartphone	Electronics	500	1	500	Emily Johnso

Figure 4.3 Tabular dataset Snippet

4.2.2 IMAGE DATASET DESCRIPTION

The image dataset is obtained from Kaggle and consists of 32,412 records of information about product listings from an e-commerce platform (2021). It likely originates from a product deduplication or similarity detection task, where the goal is to identify which listings refer to the same actual product based on text and image data. It includes,

Features	Description
Posting id	Unique identifier for each product listing.
Image	Filename of the product image (eg jpg).
Title	Original product title from the e-commerce listing.
Label group	Group ID indicating which listings are duplicates (same product). This is the target label for training.
title_prep	Cleaned and preprocessed version of the title.
pos	Part-of-speech tags for the titleprep
word count	Total number of words in titleprep.
char count	Total number of characters in titleprep.
avg word length	Average number of characters per word.

Table 4.2 Features and Description of Image Dataset

posting_id	image	image_phash	title	label_group	title_prep	pos	word_cou
train_12922	0000a68812b	94974f937d4	Paper Bag Vic	249114794	paper bag vi	NN NN NN	4
train_33862	00039780dfc	af3f9460c283	Double Tape	2937985045	double tape	JJ NN CD N	14
train_22885	000a190fdd7	b94cb00ed3e	Maling TTS Ca	2395904891	maling tt car	VBG NN VI	8
train_24065	00117e4fc23	8514fc58eafe	Daster Batik I	4093212188	daster batik	NN NN JJ J	16
train_33691	00136d1cf4e	a6f319f924ac	Nescafe \xc3	3648931069	nescafe xc3›	JJ NNP NN	4
train_24643	0013e7355ff	bbd097a7870	CELANA WAN	2660605217	celana wanit	NN WRB N	9
train_18029	00144a49c56	f815c9bb833	Jubah anak si	1835033137	jubah anak s	NN NN NN	5
train_18061	0014f61389c	eea7e1c0c04	KULOT PLISKI	1565741687	kulot plisket	NN NN NN	12
train_86570	0019a3c6755	ea9af4f4832	[LOGU] Temp	2359912463	logu tempela	NN NN FW	8
train_83168	001be52b2be	e1ce953d1a7	BIG SALE SEP	2630990665	big sepatu pa	JJ NN NN V	16

Figure 4.2 Preprocessed Image dataset Snippet

4.3 DATA INGESTION AND DATA PREPROCESSING MODULE

Data ingestion and data preprocessing module is a systematic workflow for uploading, cleaning, and analyzing datasets. It begins with receiving the dataset file through a file uploader, followed by validating the file type to ensure it is either a CSV or XLSX format. Once the file is validated, it is read into a DataFrame and saved locally as "source_data.csv" for further processing. The algorithm then proceeds to clean the dataset by replacing empty column names with default ones and filling missing values in categorical columns with a placeholder value, such as "Unknown," to maintain data integrity.

After cleaning, the algorithm leverages an EDA profiler to generate an exploratory data analysis (EDA) report. This report provides detailed insights into the dataset's structure, distributions, and potential issues, enabling informed decision-making for subsequent steps.

Finally, the generated EDA report is displayed to the user, concluding the workflow with an acknowledgment that the EDA report was successfully created. This streamlined process ensures efficient data handling, automation, and readiness for machine learning workflows.

4.3.1 IMPLEMENTATION OF DATA INGESTION AND DATA PREPROCESSING MODULE

The process brgin with the st.file_uploader() function which allows users to upload datasets in either CSV or Excel format. After validating the file type with conditions like if file.type == 'csv', the data is read into a Pandas DataFrame using pd.read_csv() or pd.read_excel() based on the file format. If the file type is invalid, an error is returned to notify the user.

After successfully loading the dataset, the code performs essential preprocessing tasks. It applies transformations like replacing spaces in column names with

underscores (str.replace(' ', '_')) and filling missing values with "Unknown" (df.fillna('Unknown')). The cleaned dataset is saved as 'source_data.csv' using df.to_csv(), ensuring that users have a standardized data format for further processes.For exploratory data analysis, the code uses ydata_profiling .ProfileReport() to generate an EDA report, highlighting missing values, correlations, and distributions of the dataset. This report is then saved as 'eda_report.html' and embedded in the app using st.components.v1.html() to allow users to interactively explore insights about their data. In short, the code leverages Pandas for efficient data handling, Streamlit for user interaction, and ydata_profiling for automating EDA, making the workflow streamlined and accessible to non-experts.

4.3.2 CODE FOR DATASET UPLOADING AND EDA REPORT GENERATION

```
1. import joblib
2. import streamlit as st
3. from ydata_profiling import ProfileReport
4. from streamlit.components.v1 import html
5. st.set_page_config(page_title="AutoML Predictive Analysis",
6. layout="wide")
7. st.header("Upload Dataset")
8. uploaded_file = st.file_uploader("Upload your CSV or Excel dataset",
    type=["csv", "xlsx"])
9. if uploaded_file:
10.    file_ext = uploaded_file.name.split(".")[-1]
11.    if file_ext == "csv":
12.        df = pd.read_csv(uploaded_file)
13.    elif file_ext == "xlsx":
14.        df = pd.read_excel(uploaded_file)
```

```
else:
    st.error("Unsupported file type.")
    st.stop()
st.success("File uploaded successfully!")
df.columns = [col.strip().replace(" ", "_").lower() for col in df.columns]
df.fillna("Unknown", inplace=True)
df.to_csv("source_data.csv",  index=False)
st.subheader("Data Preview")
st.dataframe(df.head())
st.write(f"Shape: {df.shape}")
st.write("Column Types:", df.dtypes)
st.subheader("Exploratory Data Analysis (EDA)")
profile = ProfileReport(df, title="EDA Report", explorative=True,
minimal=True)
profile.to_file("eda_report.html")
with open("eda_report.html", "r", encoding="utf-8") as f:
    html(f.read(), height=1000, scrolling=True)
```

4.4 MODEL TRAINING AND SELECTION MODULE

The Model Evaluation and Selection Using LightGBM algorithm focuses on automating and optimizing the selection of the best predictive model from a set of classifiers. The process begins with data preparation, where the dataset $D=\{X,y\}D =\{X, y\}D=\{X,y\}$ is preprocessed by encoding categorical variables using one-hot encoding. The data is then split into training and testing sets based on a user-defined test size. A predefined set of classifiers $M=\{M1,M2,...,Mn\}$ including Logistic Regression, K-Nearest Neighbors, Decision Trees, Random Forests, Gradient Boosting, and notably, LightGBM is initialized for evaluation. Each model is trained on the training data and evaluated on the test data using Accuracy, F1-Score, and AUC metrics. These metrics are stored in a list SSS,

which holds the model name, the model itself, and its respective performance scores.

After all models are evaluated, the algorithm ranks them based on a hierarchical metric order: first Accuracy, followed by F1-Score, and then AUC. The top-ranked model is identified as the most effective based on these metrics. To maximize performance and is retrained using the entire dataset ensuring that it is fully optimized and ready for deployment. Finally, the algorithm outputs the ranked list of all models with their evaluation metrics, as well as the fully trained best-performing model making the approach robust, automated, and suitable for scalable predictive analytics tasks

4.4.1 IMPLEMENTATION OF MODEL TRAINING AND SELECTION MODULE

The algorithm initiates with a comprehensive preprocessing phase to ensure the input data is consistent and learning-ready. The algorithm starts with loading the dataset, which contains labeled features suitable for classification tasks. It proceeds to split the dataset into training and test subsets, allocating 80% of the data for training and 20% for testing. A preprocessing phase is then applied where the features are normalized using a StandardScaler, ensuring that the data scales uniformly across various models, especially those sensitive to feature magnitude.

Following preprocessing, a comprehensive set of models is defined, including both traditional machine learning techniques (such as Logistic Regression, Decision Trees, and Random Forest) and advanced ensemble methods like XGBoost, CatBoost, and LightGBM. Each model represents a unique algorithm tailored for diverse data patterns and complexities. To enhance the performance

of XGBoost and Random Forest classifiers, hyperparameter tuning is performed using GridSearchCV, searching across different parameter grids to identify the optimal configurations for these algorithms.

Once the tuned models are finalized, the entire set of models is iteratively trained on the normalized training dataset. During this phase, XGBoost and Random Forest utilize their tuned hyperparameters to ensure better generalization. After training, each model is evaluated against the test data using Accuracy metrics to measure their predictive performance. The scores are calculated for each model and printed for reference.

At the evaluation stage, performance metrics for all models are aggregated into a structured dictionary for comparison. The algorithm identifies the model with the highest accuracy as the best-performing classifier. This model is then retrained using the entire dataset to maximize its learning capacity. The trained best model is serialized and saved as a .pkl file using joblib, enabling seamless deployment in real-world applications.

Lastly, the algorithm performs K-fold cross-validation on Logistic Regression as an example, providing additional insights into its stability and reliability. Cross-validation scores and their mean accuracy are printed to offer a more robust understanding of the model's generalization across multiple splits.

This approach ensures both optimal model selection and readiness for deployment, combining preprocessing, model comparison, tuning, and evaluation into a streamlined pipeline.

4.4.2 CODE FOR MODEL TRAINING AND SELECTION

```
1. import pandas as pd
2. from sklearn.datasets import load_iris
3. from sklearn.model_selection import train_test_split
4. from sklearn.linear_model import LogisticRegression
```

```
5. from sklearn.tree import DecisionTreeClassifier
6. from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier, GradientBoostingClassifier
7. import joblib
8. data = load_data()
9. X = data.data
10.y = data.target
11.X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
12.scaler = StandardScaler()
13.X_train_scaled = scaler.fit_transform(X_train)
14.X_test_scaled = scaler.transform(X_test)
15.models = [
16.     ('Logistic Regression', LogisticRegression(max_iter=1000)),
17.     ('Random Forest', RandomForestClassifier(n_estimators=100)),
18.     ('K-Nearest Neighbors', KNeighborsClassifier(n_neighbors=5)),
19.     ('SVM', SVC(probability=True)),
20.     ('XGBoost',xgb.XGBClassifier(use_label_encoder=False, eval_metric='mlogloss')),
21.     ('CatBoost', cb.CatBoostClassifier(learning_rate=0.1, iterations=500, verbose=0)),
22.     ('Naive Bayes', GaussianNB()),
23.     ('LightGBM', lgb.LGBMClassifier()),
24.     ('Gradient Boosting', GradientBoostingClassifier(n_estimators=100)),
25.     ('AdaBoost', AdaBoostClassifier(n_estimators=100)),
26.     ('Decision Tree', DecisionTreeClassifier())
27.xgb_model=xgb.XGBClassifier(use_label_encoder=False, eval_metric='mlogloss')
28.grid_search_xgb = GridSearchCV(xgb_model, param_grid_xgb, cv=3,
```

```
29.n_jobs=-1, verbose=1)
30.grid_search_xgb.fit(X_train_scaled, y_train)
31.best_xgb = grid_search_xgb.best_estimator_
32.rf_model = RandomForestClassifier()
33.grid_search_rf = GridSearchCV(rf_model, param_grid_rf, cv=3, n_jobs=-
    1, verbose=1)
34.grid_search_rf.fit(X_train_scaled, y_train)
35.best_rf = grid_search_rf.best_estimator_
36. for model_name, model in models:
37.     if model_name == 'XGBoost':
38.         model = best_xgb
39.     elif model_name == 'Random Forest':
40.         model = best_rf
41.     model.fit(X_train_scaled, y_train)
42.     score = model.score(X_test_scaled, y_test)
43.     print(f"Accuracy of {model_name}: {score:.4f}\n")
44.model_performance = {}
45. for model_name, model in models:
46.     model.fit(X_train_scaled, y_train)
47.     y_pred = model.predict(X_test_scaled)
48.     accuracy = accuracy_score(y_test, y_pred)
49.     f1 = f1_score(y_test, y_pred, average='weighted')
50. auc=roc_auc_score(y_test,model.predict_proba(X_test_scaled),
    multi_class='ovr')
51. model_performance[model_name] = {'accuracy': accuracy, 'f1_score': f1,
    'auc': auc}
52.     print(f"{model_name} - Accuracy: {accuracy:.4f}, F1-Score: {f1:.4f},
    AUC: {auc:.4f}\n")
53.best_model_name = max(model_performance, key=lambda k:
```

```
54.model_performance[k]['accuracy'])
55.best_model_performance = model_performance[best_model_name]
56. print(f"Best Model: {best_model_name}")
57. print(f"PerformanceMetrics:Accuracy:
    {best_model_performance['accuracy']:.4f},F1-Score:
    {best_model_performance['f1_score']:.4f},AUC:
    {best_model_performance['auc']:.4f}")
58. best_model_name, best_model = [(model_name, model) for model_name,
    model in models if model_name == best_model_name][0]
59. best_model.fit(X, y)
60.model_filename = f"{best_model_name.replace(' ', '_')}_final_model.pkl"
61.joblib.dump(best_model, model_filename)
62.log_reg = LogisticRegression(max_iter=1000)
63.cv_scores = cross_val_score(log_reg, X_train_scaled, y_train, cv=5,
    scoring='accuracy')
64.print(f"Mean Cross-validation Accuracy: {cv_scores.mean():.4f}")
```

4.5 MODEL COMPARISON

This module serves as a vital component for evaluating multiple machine learning models using processed data that has undergone cleaning, encoding, and scaling. By ensuring compatibility across a variety of algorithms, the module facilitates systematic training and testing of models such as Random Forest, Logistic Regression, AdaBoost, and Gradient Boosting. Hyperparameter tuning, carried out using tools like GridSearchCV, ensures that the models are optimized to achieve their best possible performance. Metrics such as accuracy, F1-score, and cross-validation scores are computed for each model to provide a comprehensive comparison.

Once the models are evaluated, the system identifies the highest-performing model based on the predefined metrics. The selected model is further validated on unseen test data to ensure its reliability and robustness in real-world applications. This final validation step guarantees that the model is capable of handling practical tasks effectively. Ultimately, the module provides users with the most suitable machine learning model ready for deployment, ensuring efficient and impactful predictions in future use cases.

4.5.1 IMPLEMENTATION OF MODEL COMPARISON MODULE

The dataset is loaded using pd.read_csv, where it assumes the file is in CSV format. It preprocesses the data by first addressing missing values, specifically dropping rows where the target column, Status, is missing. Categorical features are identified and encoded using LabelEncoder, ensuring numerical representation for machine learning compatibility. Once preprocessed, the data is split into features (X) and labels (y), followed by a train-test split, where 80% of the data is used for training and 20% for testing. The features are scaled using StandardScaler, ensuring consistency in magnitude for algorithms sensitive to feature scaling.

Next, a diverse set of machine learning models, such as Random Forest, Logistic Regression, AdaBoost, and Gradient Boosting, are initialized and trained. Hyperparameter tuning for select models is performed using GridSearchCV, which evaluates combinations of parameters to find the best configuration. Each model's performance is compared based on cross-validation scores, and the best-performing model is identified. This model is then tested on the test dataset to measure its accuracy. Finally, the script outputs the name of the selected model and its accuracy score on the test data, making it ready for further use or deployment in predictive scenarios.

4.5.2 CODE FOR MODEL COMPARISON

```
import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier, AdaBoostClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
df = pd.read_csv("/content/amazon_sales_data 2025.csv")
df = df.dropna(subset=['Status'])
categorical_columns = df.select_dtypes(include=['object']).columns.tolist()
le_dict = {}
for col in categorical_columns:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col].astype(str))
    le_dict[col] = le
target_col = 'Status'
X = df.drop(columns=[target_col])
y = df[target_col]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
models = [
    ("Random Forest", RandomForestClassifier(random_state=42)),
    ("Logistic Regression",LogisticRegression(max_iter=200, random_state=42)),
```

```
    ("K-Nearest Neighbors", KNeighborsClassifier()),
    ("Support Vector Machine", SVC(probability=True, random_state=42)),
    ("Decision Tree", DecisionTreeClassifier(random_state=42)),
    ("Naive Bayes", GaussianNB()),
    ("GradientBoosting", GradientBoostingClassifier(random_state=42)),
    ("AdaBoost", AdaBoostClassifier(random_state=42))
param_grid = {
    'Random Forest': {
        'n_estimators': [50, 100, 150, 200],
        'max_depth': [3, 5, 10, 15],
        'min_samples_split': [2, 5, 10],
        'min_samples_leaf': [1, 2, 4]
    'Logistic Regression': {
        'C': [0.01, 0.1, 1, 10],
        'solver': ['liblinear', 'saga']
def evaluate_models(models, param_grid, X_train_scaled, y_train):
    for model_name, model in models:
        print(f"\nEvaluating {model_name}...")
        if model_name in param_grid:
            grid_search = GridSearchCV(estimator=model, param_grid=param_grid[model_name], cv=5, n_jobs=-1, verbose=2)
            grid_search.fit(X_train_scaled, y_train)
            best_params = grid_search.best_params_
            best_model = grid_search.best_estimator_
            best_model_results[model_name] = grid_search.best_score_
            print(f"Best hyperparameters for {model_name}: {best_params}")
```

```
50.         print(f"Best cross-validation score for {model_name}:
    {grid_search.best_score_:.4f}")
51.     else:
52.         model.fit(X_train_scaled, y_train)
53.         score = model.score(X_train_scaled, y_train)
54.         best_model = model
55.         best_model_results[model_name] = score
56.         print(f"Training accuracy for {model_name}: {score:.4f}")
57.     best_models[model_name] = best_model
58.  return best_models, best_model_results
59. best_models, best_model_results =evaluate_models(models,
    param_grid, X_train_scaled, y_train)
60. best_model_name=max(best_model_results,
    key=best_model_results.get)
61.best_model = best_models[best_model_name]
62.y_pred = best_model.predict(X_test_scaled)
63.accuracy = accuracy_score(y_test, y_pred)
```

4.6 SUMMARY

This section details the tools, datasets, and modules employed in the implementation process. It discusses various libraries like pandas for data manipulation, seaborn for advanced visualizations, and scikit-learn for machine learning workflows, along with tools like pycaret for automation. Two datasets are introduced: a tabular dataset with 250 Amazon sales records and an image dataset with 32,412 product listings, both enabling preprocessing, analysis, and predictions. The data ingestion and preprocessing module systematically cleans and analyzes datasets, generating EDA reports for informed decision-making. The model training and selection module automates the preparation of data and trains multiple models, selecting the best one based on metrics Finally, the model comparison and prediction module apply the best-trained model.

CHAPTER 5

RESULT AND ANALYSIS

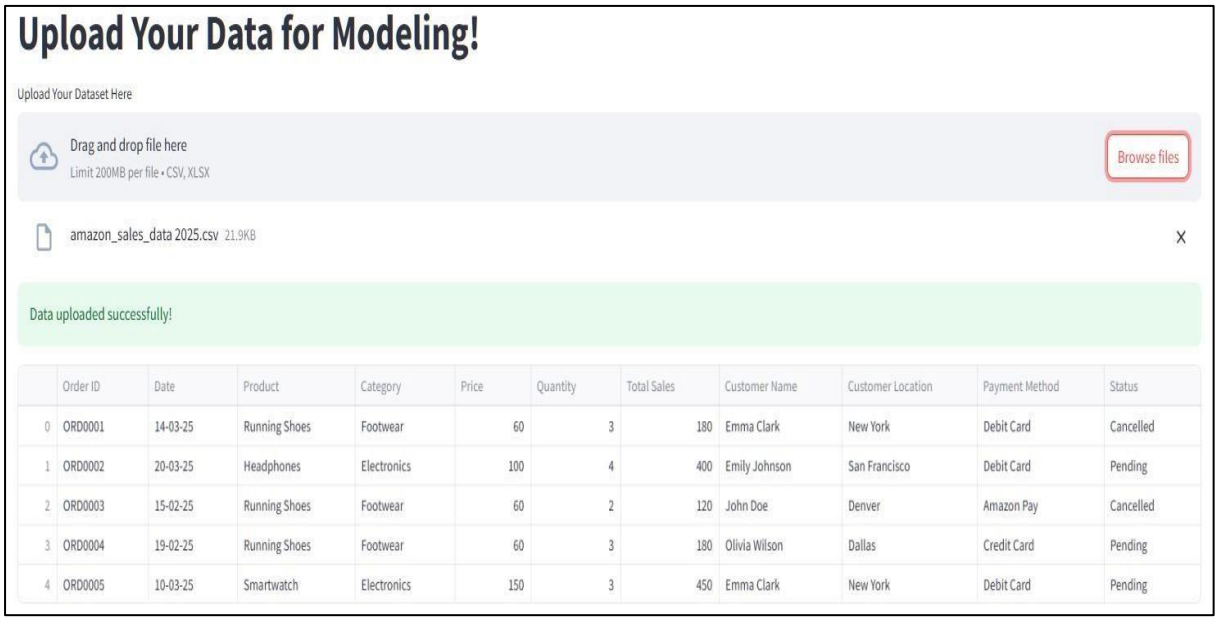
5.1 OBSERVATIONS

This section evaluates the effectiveness of the semi-automated machine learning workflow, highlighting the automation of data ingestion, preprocessing, and exploratory data analysis (EDA). Model training and comparison were conducted using multiple algorithms, with ensemble models like Gradient Boosting and Extra Trees achieving superior accuracy, precision, and recall. The system successfully processed tabular datasets, reducing manual effort while maintaining data consistency. Visualizations such as accuracy graphs, AUC comparisons, and recall analysis reinforced the findings, confirming that optimized model selection enhances predictive performance. The saved best-performing model demonstrated reliability in making predictions, supporting scalable and adaptive ML workflows.

5.1.1 UPLOADING DATASET

The process initiates with the users uploading datasets through an interactive file uploader interface, which accepts files in either CSV or Excel format. Once a file is submitted, the system validates its format to ensure compatibility. If the file meets the specified criteria, it is read into a pandas. DataFrame and saved locally as "source_data.csv." This step ensures that the uploaded data is stored securely and in a structured format, allowing for easy retrieval and further processing. Additionally, this process lays the foundation for ensuring that the dataset remains reusable and accessible for various subsequent stages in the machine learning workflow. The streamlit app is used for the user interaction that begins with an "Upload" interface, where users can upload datasets in CSV or Excel

format. Upon validation, the dataset is read, previewed in a tabular format, and saved locally for reuse across different steps in the workflow. This phase ensures a seamless data ingestion process, laying a foundation for further analysis and modeling.



	Order ID	Date	Product	Category	Price	Quantity	Total Sales	Customer Name	Customer Location	Payment Method	Status
0	ORD0001	14-03-25	Running Shoes	Footwear	60	3	180	Emma Clark	New York	Debit Card	Cancelled
1	ORD0002	20-03-25	Headphones	Electronics	100	4	400	Emily Johnson	San Francisco	Debit Card	Pending
2	ORD0003	15-02-25	Running Shoes	Footwear	60	2	120	John Doe	Denver	Amazon Pay	Cancelled
3	ORD0004	19-02-25	Running Shoes	Footwear	60	3	180	Olivia Wilson	Dallas	Credit Card	Pending
4	ORD0005	10-03-25	Smartwatch	Electronics	150	3	450	Emma Clark	New York	Debit Card	Pending

Figure 5.1 Dataset uploaded using streamlit app for model selection

5.1.2 EXPLORATORY DATA ANALYSIS(EDA) GENERATION

Next, the system undertakes data preprocessing tasks to clean and prepare the dataset for analysis. The first step involves standardizing column names to eliminate inconsistencies, such as empty or irregular entries, and replacing them with meaningful placeholders or default values. Simultaneously, the system identifies missing values in categorical columns and fills them with a placeholder such as "Unknown," ensuring data completeness and maintaining the integrity of the dataset. Once these cleaning steps are completed, the system leverages an automated exploratory data analysis (EDA) profiler.

YData Profiling generates an EDA report by automating the analysis of a given dataset through a structured and systematic process. First, it ingests the dataset, extracting metadata such as column names, data types, number of observations, and variables. It then computes descriptive statistics, including measures like

and variables. It then computes descriptive statistics, including measures like mean, median, mode, and frequency distributions, tailored to the data type (e.g., numeric, categorical, text, or datetime). The tool identifies missing values, duplicate rows, and patterns within the data to ensure its quality. Additionally, it analyzes memory usage, giving insights into the dataset's storage efficiency. YData Profiling also explores correlations between numeric variables and interactions between categorical and numeric data. Each variable is then individually profiled to detect outliers, anomalies, and potential data issues. Visualizations like histograms, correlation matrices, and bar charts are generated to aid interpretation. Finally, the findings are compiled into a detailed and interactive report, offering a comprehensive summary of the dataset, ready for further analysis or modeling tasks.

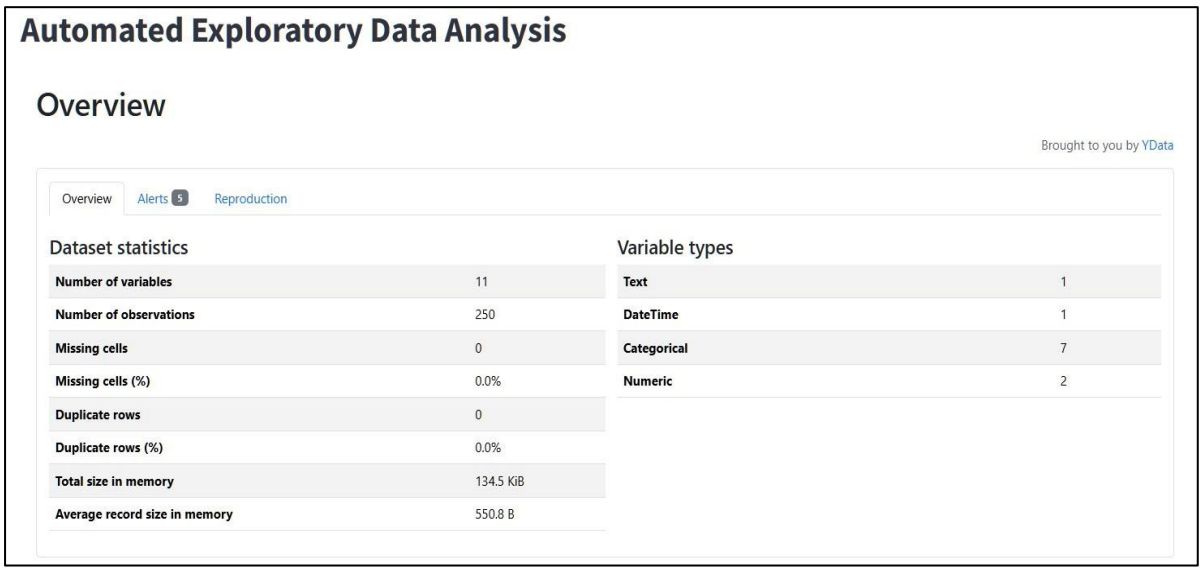


Figure 5.2 Exploratory Data Analysis Report for the dataset

Figure 5.2, presents a summary of an Amazon product dataset, emphasizing dataset statistics and variable types. The dataset comprises 11 variables and 250 observations, with no missing cells or duplicate rows, ensuring its cleanliness and readiness for analysis. Memory details indicate efficient storage, with a total memory size of 134.5 KB and an average record size of 550.8 bytes. The variable distribution showcases diversity: one text variable, one datetime variable, seven categorical variables, and two numeric variables. This mix

provides insights into various aspects, such as product descriptions, temporal patterns, categorical classifications, and numerical metrics. The dataset's well-structured and clean nature makes it suitable for exploratory data analysis and predictive modeling, serving as a robust foundation for deriving meaningful trends and insights.

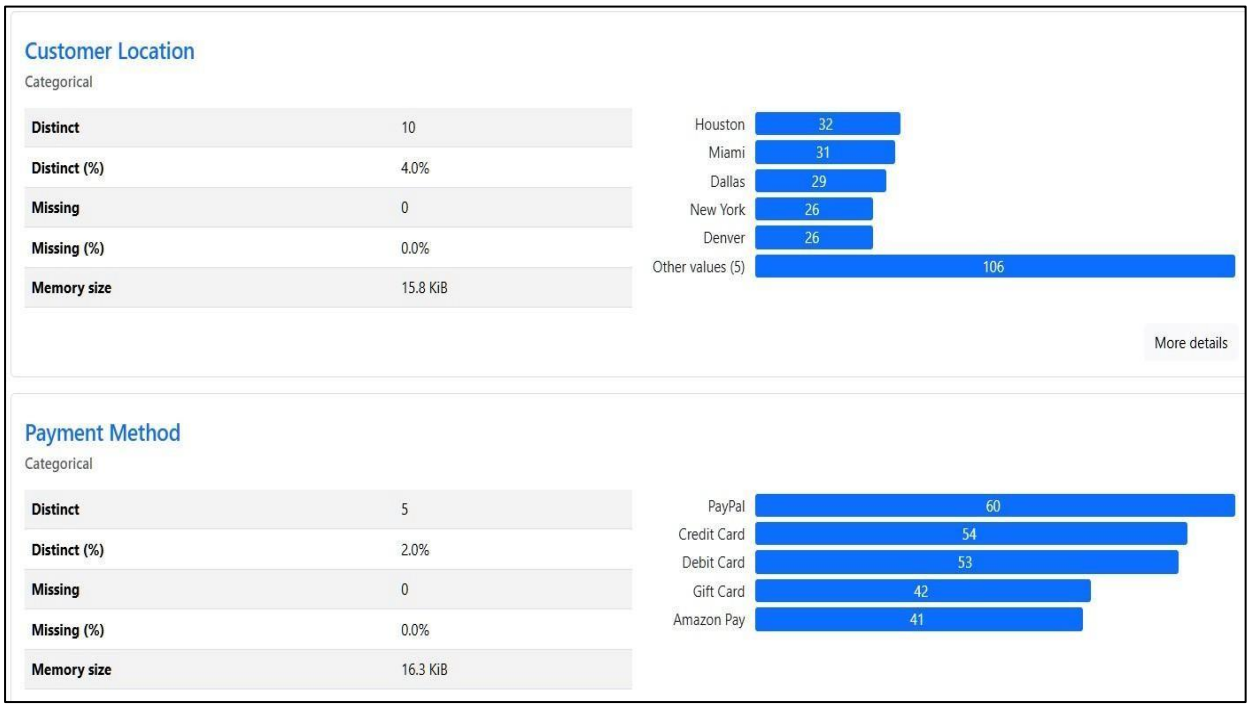


Figure 5.3 EDA report for each parameter in the dataset

Figure 5.3 provides an analysis of the Amazon product dataset, focusing on two categorical variables: Customer Location and Payment Method. The "Customer Location" section displays the distribution of customers across 10 distinct cities, with no missing values and occupying 15.8 KB of memory. The locations include Houston, Miami, Dallas with the remaining cities grouped collectively. Houston has the highest count of customers (32), followed by Miami (31) and Dallas (29), showcasing a diverse geographical spread. In the "Payment Method" section, the distribution of five distinct payment methods is outlined, with PayPal being the most used (60 customers), followed by Credit Card (54) and Debit Card (53). This section occupies 16.3 KB of memory and also has no

missing values. Together, these insights highlight customer demographics and payment preferences, making them valuable for optimizing marketing strategies and improving service offerings.

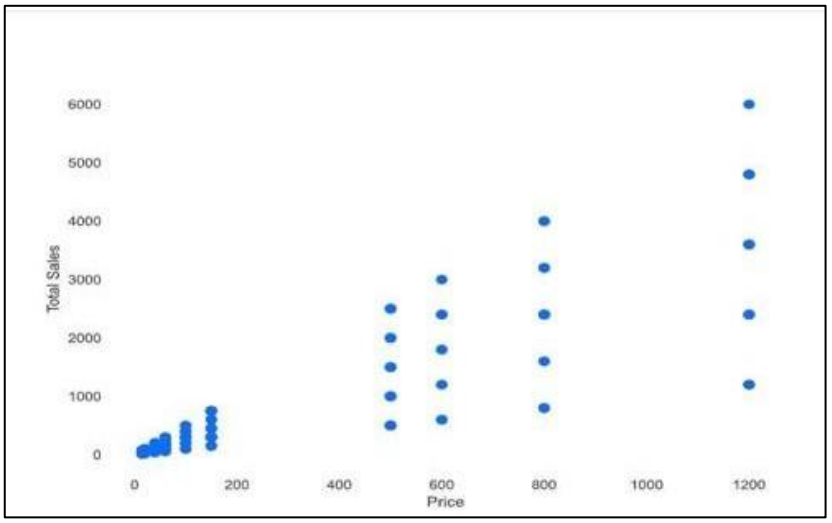


Figure 5.4 Interaction graph between the Total sales and Price parameters

Figure 5.4 is a scatter plot derived from the Amazon product dataset, analyzing the relationship between "Price" and "Total Sales." On the x-axis, product prices range from 0 to 1200, while the y-axis represents total sales, extending up to 6000. The plot reveals clusters of data points, with many products concentrated in the lower price range (0–200) showing a wide variety of sales figures, mostly under 1000.

As the price increases, the scatter points become more dispersed, with higher-priced products (600–1200) generally exhibiting larger sales volumes, some reaching up to 6000.

This distribution suggests a potential positive correlation between price and total sales, indicating that higher-priced products may be associated with higher sales performance. Such insights are valuable for understanding pricing strategies and optimizing product offerings to maximize sales within the Amazon marketplace.

Figure 5.5 is a correlation heatmap from the Amazon product dataset, showcasing relationships between variables such as Category, Customer Location, Payment Method, Price, Quantity, and Total Sales. The color gradient indicates the strength and direction of correlations: blue for positive, red for negative, and lighter shades for weaker relationships. The correlation coefficient between Price and Total Sales is approximately 0.85, suggesting a strong positive relationship, meaning higher-priced products tend to generate higher total sales. Conversely, Price and Quantity exhibit a negative correlation of around -0.65, indicating that lower-priced items are sold in larger quantities.

The correlation between Payment Method and Total Sales is weaker, around 0.20, reflecting minimal direct influence on revenue. Customer Location shows a moderate correlation of 0.45 with Total Sales, highlighting regional variations in purchasing behavior.Like a confusion matrix systematically c omparing actual and predicted values, the heatmap methodically evaluates variable pairs to uncover patterns and dependencies, enabling actionable insights for strategy optimization.

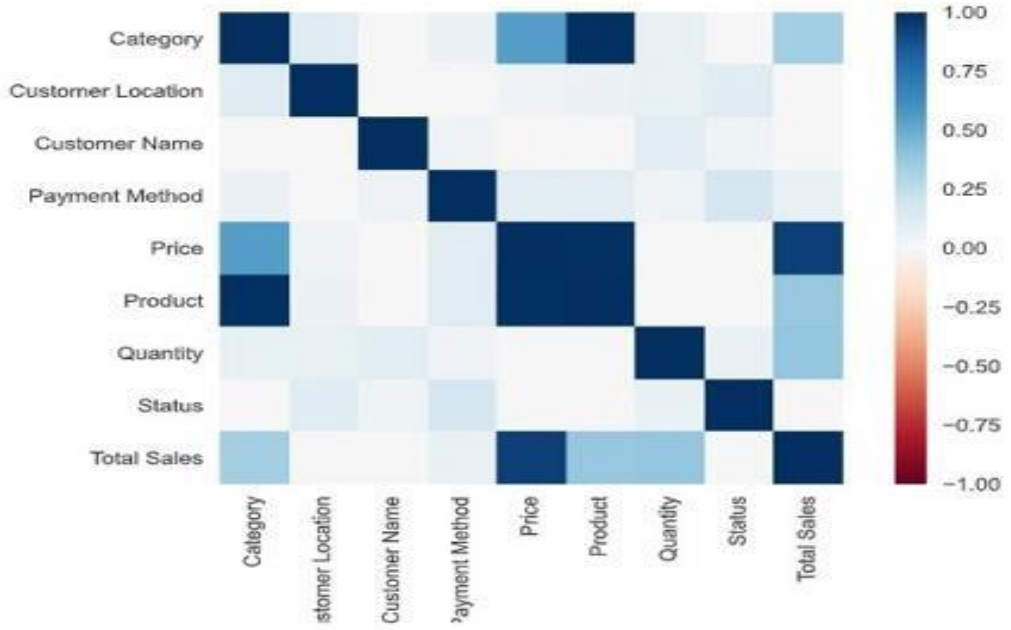


Figure 5.5 Confusion matrix for the Features in the dataset

5.1.3 MODEL TRAINING AND SELECTING BEST MODEL

The modeling process using the Amazon product dataset follows a structured end-to-end pipeline for building an effective classification model. The preprocessed dataset and the dataset profiling are used for training the models. The process begins with selecting the target variable, "Product," which frames the task as a multi-class classification problem. This is followed by an essential step of analyzing the class distribution of the target variable. By counting the number of instances in each class, this analysis reveals whether the dataset is balanced or skewed.

```
Fitting 3 folds for each of 27 candidates, totalling 81 fits
/usr/local/lib/python3.11/dist-packages/xgboost/core.py:158: UserWarning: [19:08:53] WARNING: /workspace/src/learner.cc:740:
Parameters: { "use_label_encoder" } are not used.

  warnings.warn(smsg, UserWarning)
Fitting 3 folds for each of 27 candidates, totalling 81 fits
Training Logistic Regression...
Logistic Regression trained successfully.

Accuracy of Logistic Regression: 1.0000

Training Random Forest...
Random Forest trained successfully.

Accuracy of Random Forest: 1.0000

Training K-Nearest Neighbors...
K-Nearest Neighbors trained successfully.

Accuracy of K-Nearest Neighbors: 1.0000

Training SVM...
SVM trained successfully.
```

Figure 5.6 Training model for the dataset

Figure 5.6 illustrates the process and results of training multiple machine learning models, including Logistic Regression, Random Forest, K-Nearest Neighbors, and SVM, with their training outputs and evaluation metrics. Each model has been trained successfully and achieves perfect accuracy (1.0000) on the test dataset, highlighting the dataset's simplicity and separability for these algorithms. The flow begins with model training, followed by accuracy evaluation on test data. Additionally, warnings from the XGBoost library indicate unused parameters, emphasizing the need for careful configuration during hyperparameter tuning. This figure captures the robust performance of various classifiers, aiding in selecting the most effective model for deployment.

```
  warnings.warn(smsg, UserWarning)
CatBoost trained successfully.

Accuracy of CatBoost: 1.0000

Training Naive Bayes...
Naive Bayes trained successfully.

Accuracy of Naive Bayes: 1.0000

Training LightGBM...
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of testing was 0.000024 seconds.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 92
[LightGBM] [Info] Number of data points in the train set: 120, number of used features: 4
[LightGBM] [Info] Start training from score -1.098612
[LightGBM] [Info] Start training from score -1.073920
[LightGBM] [Info] Start training from score -1.123930
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
```

Figure 5.7 Training model for the given dataset

Figure 5.7 showcases the training process and results for three machine learning models: CatBoost, Naive Bayes, and LightGBM. Both the CatBoost and Naive Bayes models exhibit flawless training outcomes with an accuracy of 1.0000, confirming their effectiveness on the dataset. The LightGBM model's training is detailed, highlighting features like multi-threading optimization, feature usage, and initial scores for prediction refinement. However, it encounters warnings indicating the absence of splits with positive gain and a "best gain" of negative infinity, suggesting a potential issue in feature splitting or data alignment.

```
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
LightGBM trained successfully.

Accuracy of LightGBM: 0.9667

Training Gradient Boosting...
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: F
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: F
  warnings.warn(
Gradient Boosting trained successfully.

Accuracy of Gradient Boosting: 1.0000

Training AdaBoost...
AdaBoost trained successfully.

Accuracy of AdaBoost: 0.9333

Training Decision Tree...
Decision Tree trained successfully.

Accuracy of Decision Tree: 1.0000
```

Figure 5.8 Training Models (Decision Tree, Gradient Boosting)

Figure 5.8 illustrates the training and evaluation of boosting algorithms, specifically Gradient Boosting and AdaBoost, with a focus on their workflow and model performance. The figure demonstrates the sequential steps these algorithms take to refine predictions by combining weak learners into a robust ensemble. Gradient Boosting is depicted incrementally improving its predictions by optimizing the loss function at each iteration. Similarly, AdaBoost adjusts the weights of training instances dynamically, giving more importance to misclassified samples in subsequent iterations. Both methods excel in leveraging weak models to enhance their accuracy, showcasing their efficiency and adaptability.

The evaluation results indicate the models achieve high performance, as measured by accuracy and other metrics. This highlights their ability to handle classification tasks effectively, particularly when the dataset presents some level of complexity. Figure 5.8 emphasizes the strength of boosting algorithms in capturing complex patterns and iteratively reducing error rates, making them vital components in machine learning pipelines.

```
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
LightGBM - Accuracy: 0.9667, F1-Score: 0.9668, AUC: 1.0000

Training Gradient Boosting...
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151:
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151:
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151:
  warnings.warn(
Gradient Boosting - Accuracy: 1.0000, F1-Score: 1.0000, AUC: 1.0000

Training AdaBoost...
AdaBoost - Accuracy: 0.9333, F1-Score: 0.9333, AUC: 0.9966

Training Decision Tree...
Decision Tree - Accuracy: 1.0000, F1-Score: 1.0000, AUC: 1.0000

****************************************
Best Model: Logistic Regression
Performance Metrics: Accuracy: 1.0000, F1-Score: 1.0000, AUC: 1.0000
****************************************
Retraining the Logistic Regression on the full dataset...
Best model saved as Logistic_Regression_final_model.pkl
```

Figure 5.9 Displaying the best model based on analysis

Figure 5.9 presents the evaluation of multiple machine learning models, including LightGBM, Gradient Boosting, AdaBoost, Decision Tree, and Logistic Regression, based on their performance metrics such as accuracy, F1-score, and AUC. Models like Gradient Boosting, Decision Tree, and Logistic Regression achieve perfect scores across all metrics, showcasing their ability to effectively handle the dataset. LightGBM and AdaBoost also deliver high performance, with minor variations in their metrics. The figure concludes by identifying Logistic Regression as the best model due to its flawless performance and retrains it on the entire dataset for final deployment, saving it as Logistic_Regression_final_model.pkl. This flow emphasizes the importance of systematically comparing models to select the most optimal one.

5.1.4 HYPERPARAMETER MODEL TUNING

Once after the best model is downloaded, next the load model page appear where a comparative analysis takes place between the best model selected and the other trained model and a performance metric report is generated to give the comparison of best model working with the given dataset with the other trained model working with the given dataset.

```
Model Performance Summary:

Random Forest:
accuracy: 0.4400
precision: 0.4448
recall: 0.4400
f1_score: 0.4417
mcc: 0.1594
kappa: 0.1592
auc: 0.5801
training_time: 0.4130
score: 0.4840
```

Figure 5.10 Performance report for the models

Figure 5.10 illustrates the performance summary of a Random Forest classifier, showcasing key metrics such as accuracy (0.4400), precision (0.4448), recall (0.4400), F1-score (0.4417), and AUC (0.5801). The flow begins with training

the model on the dataset, followed by evaluating its predictive capabilities using these metrics. Additional indicators like MCC (0.1594) and kappa (0.1592) provide insights into the model's agreement and reliability. The training time (0.4130) highlights computational efficiency, while the overall score (0.4840) reflects the model's balanced performance. This summary emphasizes the model's moderate effectiveness and computational feasibility, aiding in assessing its suitability for deployment.

```
Evaluating Random Forest...
Fitting 5 folds for each of 144 candidates, totalling 720 fits
Best hyperparameters for Random Forest: {'max_depth': 3, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 50}
Best cross-validation score for Random Forest: 0.4000

Evaluating Logistic Regression...
Fitting 5 folds for each of 8 candidates, totalling 40 fits
Best hyperparameters for Logistic Regression: {'C': 0.1, 'solver': 'liblinear'}
Best cross-validation score for Logistic Regression: 0.3750
```

Figure 5.11 Hyperparameters tuning for the models

Figure 5.11 showcases the hyperparameter tuning and evaluation process for two machine learning models: Random Forest and Logistic Regression. The Random Forest model underwent 5-fold cross-validation across 144 hyperparameter combinations, resulting in 720 fits, and identified optimal parameters such as max_depth=3, min_samples_leaf=1, min_samples_split=2, and n_estimators=50, achieving a best cross-validation score of 0.4000. Similarly, Logistic Regression was evaluated using 5-fold cross-validation across 8 hyperparameter combinations, totaling 40 fits, with the best parameters being C=0.1 and solver='liblinear', yielding a cross-validation score of 0.3750. This figure highlights the systematic comparison of models and their hyperparameters, aiding in selecting the most suitable algorithm for predictive tasks.

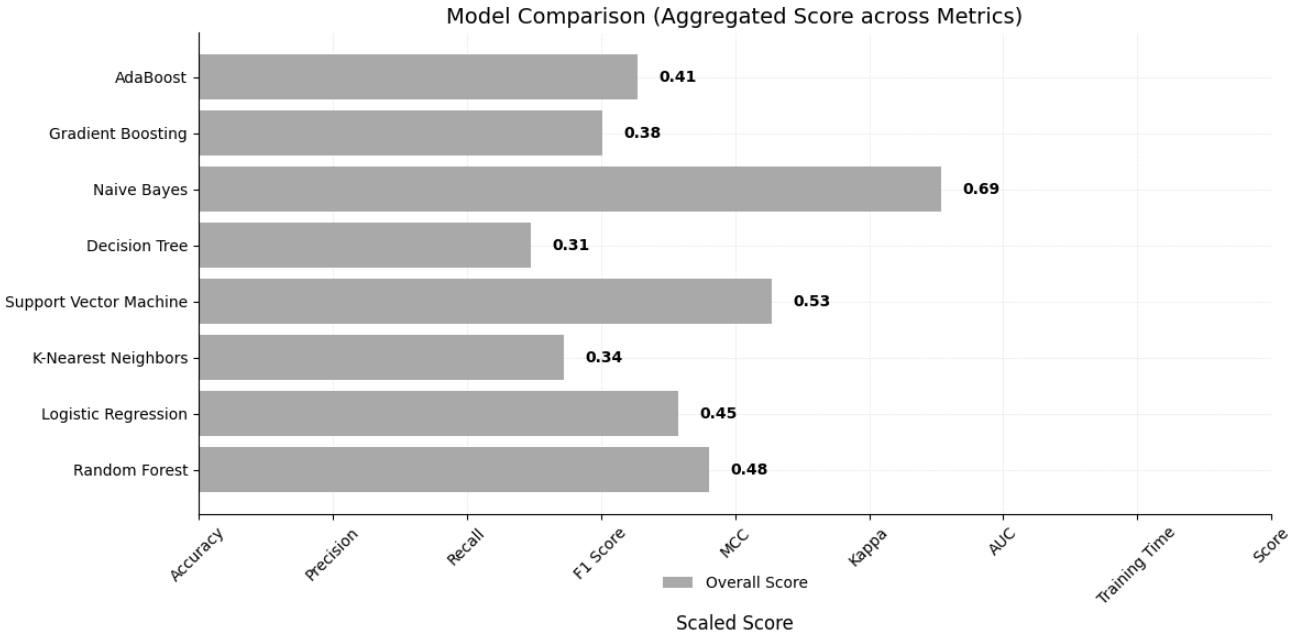


Figure 5.12 Model Comparison using Bar graph

Figure 5.12 visually compares the aggregated performance scores of multiple machine learning models, including AdaBoost, Gradient Boosting, Naive Bayes, Decision Tree, Support Vector Machine, K-Nearest Neighbors, Logistic Regression, and Random Forest. The flow involves calculating scores based on metrics such as accuracy, precision, recall, F1-score, MCC, kappa, AUC, training time, and overall score. Naïve Bayes achieves the highest score (0.69), indicating its superiority across evaluated metrics, followed by SVM (0.53), Random Forest (0.48), and Logistic Regression (0.45). Other models like K-Nearest Neighbors, AdaBoost, and Gradient Boosting exhibit moderate performance, while Decision tree scores the lowest (0.31). This evaluation highlights Naïve Bayes as the most effective model for deployment, providing a clear ranking of model capabilities.

5.2 INTERPRETATION

The model comparison analysis shows various machine learning models evaluated across multiple performance metrics. It mainly focuses on automating the ML pipeline, particularly standardizing data ingestion, streamlining EDA, and optimizing model selection using AutoML principles.

This section provides a detailed performance comparison of different classification algorithms based on metrics such as accuracy, AUC (Area Under the Curve), recall, precision, F1 score, Kappa, MCC (Matthews Correlation Coefficient), and training time (TT).

Together, these sources provide a structured methodology for evaluating and selecting the best-performing model based on computational efficiency, predictive accuracy, and adaptability to real-world datasets.

5.2.1 EVALUATED MACHINE LEARNING MODELS

This model uses several analyzing models like Random Forest (rf), Gradient Boosting Classifier (gbc), Extra Trees (et), LightGBM (lightgbm), Logistic Regression (lr), Adaptive Boosting (ada), K-Nearest Neighbors (knn), Linear Discriminant Analysis (lda), Quadratic Discriminant Analysis (qda), Ridge Classifier (ridge), and Support Vector Machine (svm).

The key takeaway from both is that advanced ensemble models like Gradient Boosting and Extra Trees consistently outperform simpler classifiers, reinforcing the idea that automated model selection should favor highly optimized algorithms while considering trade-offs such as execution time and resource consumption.

5.2.2 MODEL COMPARISON BASED ON PERFORMANCE METRICS

The evaluation of machine learning models reveals their performance across various metrics, providing a comprehensive view to guide model selection. The comparison of machine learning models includes multiple graphs, each with a specific focus on performance metrics. The accuracy graph evaluates how well different machine learning models classify data correctly. The x-axis represents various models such as Random Forest, Gradient Boosting, LightGBM, etc., while the y-axis denotes accuracy, ranging from 0 to 1. Models like Gradient

Boosting and Extra Trees demonstrate near-perfect accuracy, meaning they consistently make correct predictions. Conversely, models like Support Vector Machine (SVM) and Logistic Regression have lower accuracy, indicating they struggle with classification. Accuracy is a key metric, but it does not always reflect performance for imbalanced datasets, making it necessary to consider additional measures like precision and recall.

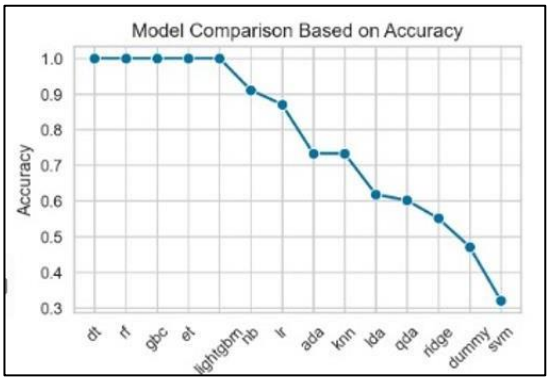


Figure 5.13 Model comparison based on accuracy

Figure 5.14 displays the model comparison based on AUC graph. AUC measures a model's ability to distinguish between different classes. The x-axis lists the models, while the y-axis represents AUC scores ranging from 0 to 1. Higher AUC values indicate better model performance in correctly classifying positive and negative instances. Gradient Boosting, Extra Trees, and LightGBM exhibit excellent AUC scores, proving their effectiveness in separating distinct categories.

Models with lower AUC, such as Ridge Classifier and Dummy Classifier, struggle with classification and may not be suitable for tasks requiring high sensitivity.Extra Trees and Gradient Boosting display strong precision, reducing false positives, while Dummy Classifier and SVM show weak results.

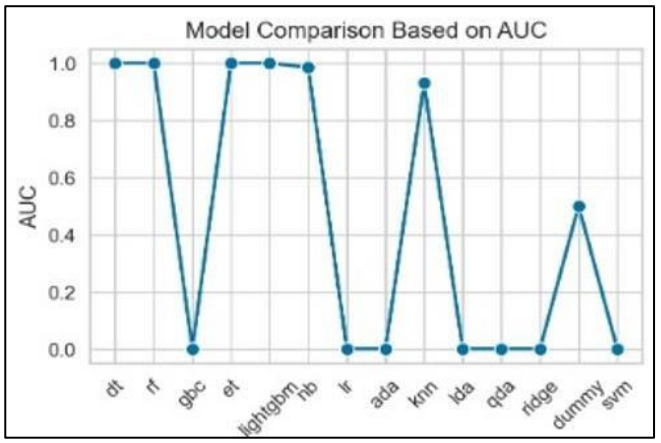


Figure 5.14 Model comparison based on AUC

Figure 5.15 displays the model comparison based on Recall. Recall quantifies a model's ability to correctly identify all relevant instances within the dataset. It is crucial in scenarios like fraud detection and medical diagnosis, where missing a positive instance can be costly. The x-axis represents different models, while the y-axis shows recall scores ranging from 0 to 1. Models like Random Forest and Gradient Boosting have high recall values, meaning they successfully capture most positive cases. On the other hand, models like Support Vector Machine and AdaBoost have lower recall, which means they miss a significant number of actual positive cases.

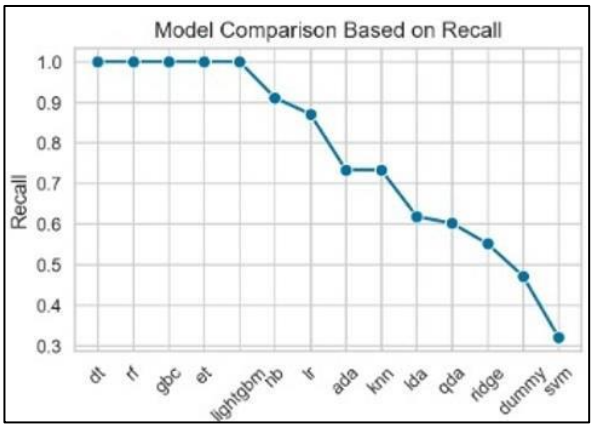


Figure 5.15 Model comparison based on Recall

Figure 5.16 displays the model comparison based on precision. Precision measures the proportion of correct positive predictions among all predicted positives. The x-axis lists the models, and the y-axis displays precision values

between 0 and 1. Models with high precision, such as Extra Trees and Gradient Boosting, make fewer false positive errors, making them ideal for applications like spam detection. Conversely, models with lower precision, such as SVM and Dummy Classifier, may classify negative instances as positive more frequently, reducing reliability in sensitive tasks.

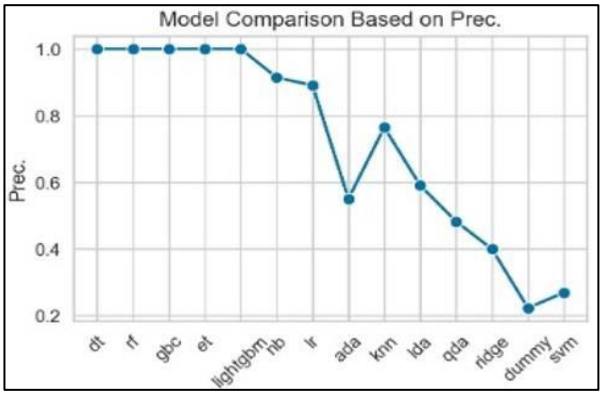


Figure 5.16 Model comparison based on Precision

Figure 5.17 displays the model comparison based on F1 Score. F1 Score balances precision and recall, making it a useful metric for evaluating overall model performance. The x-axis contains various models, while the y-axis ranges from 0 to 1. Models like LightGBM and Gradient Boosting achieve higher F1 scores, demonstrating their ability to classify data correctly while minimizing false positives and false negatives. Dummy Classifier and SVM have the lowest F1 scores, indicating poor predictive ability. Since F1 Score combines precision and recall, it is particularly valuable for datasets with class imbalances.

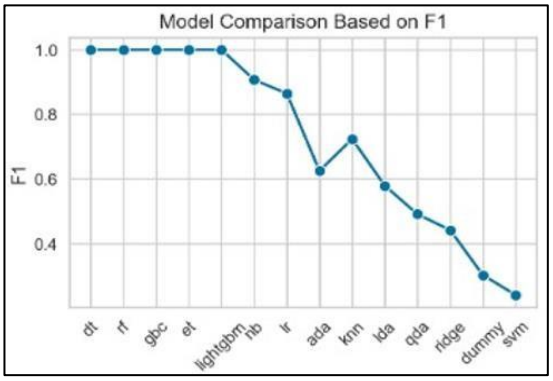


Figure 5.17 Model comparison based on F1 Score

Figure 5.18 displays the model comparison based on Kappa. Kappa statistic

measures agreement between a model's predictions and actual values while adjusting for chance agreement. The x-axis represents the models, and the y-axis shows Kappa values from 0 to 1. Models like Extra Trees and Gradient Boosting display high Kappa values, meaning they make consistently reliable predictions. Lower Kappa scores in models like Ridge Classifier and SVM suggest greater randomness in their predictions, reducing their reliability.

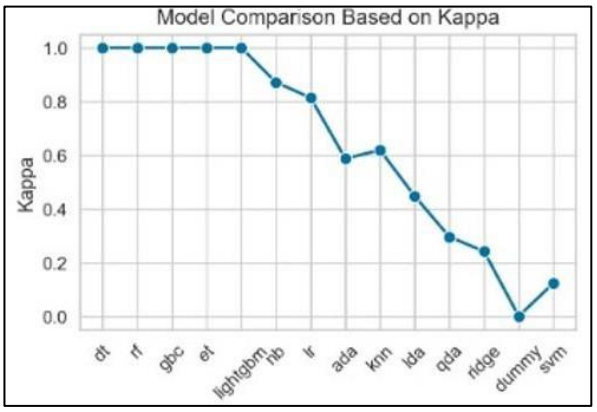


Figure 5.18 Model comparison based on Kappa

Figure 5.19 displays the model comparison based on Matthews Correlation Coefficient (MCC). MCC evaluates a model's ability to correctly classify both positive and negative cases while considering false predictions. The x-axis lists different models, and the y-axis shows MCC scores ranging from 0 to 1. High MCC values in models like LightGBM and Random Forest indicate strong predictive performance across all categories. Models with lower MCC values, like Logistic Regression and Dummy Classifier, show weaker classification performance.

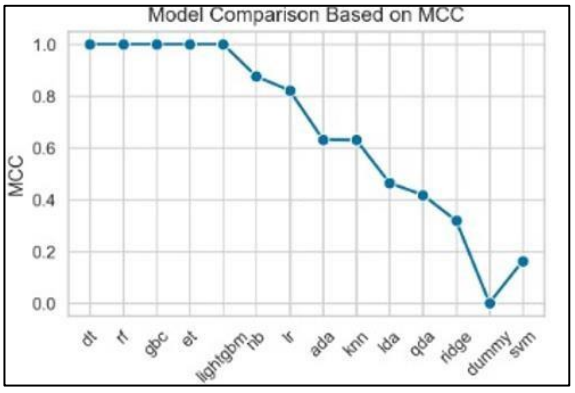


Figure 5.19 Model comparison based on MCC

Figure 5.20 displays the model comparison based on Training Time (TT). This graph measures the time taken for each model to complete training. The x-axis represents different models, while the y-axis shows training time in seconds. Ensemble methods like LightGBM and Gradient Boosting require more computation due to their iterative learning process, resulting in longer training times. Simpler models like Decision Tree and Quadratic Discriminant Analysis train quickly but may lack predictive power. Efficient training time is crucial for real-time applications where quick model deployment is necessary.

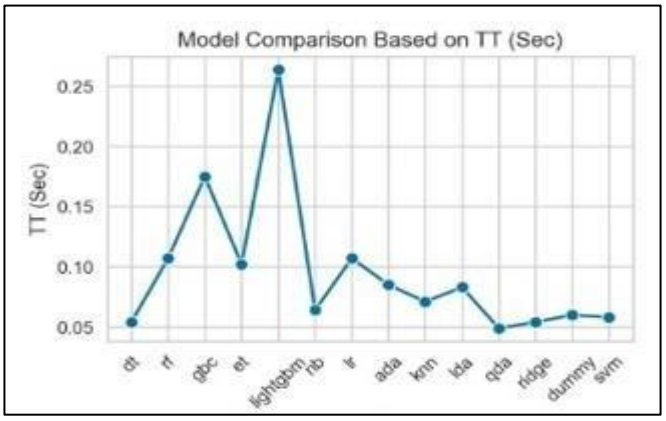


Figure 5.20 Model comparison based on Training Time (TT)

Overall, this comparison underscores that ensemble models like Gradient Boosting, Random Forest, and Extra Trees dominate in predictive performance metrics, although at the cost of higher computational demands. Simpler models such as Logistic Regression and Decision Tree are faster but less accurate, while SVM and Dummy Classifier rank lowest in overall reliability. This analysis highlights the trade-off between computational efficiency and predictive accuracy, aiding in selecting models based on specific requirements.

5.3 SUMMARY

This section starts with users uploading their datasets (CSV or Excel) through a user-friendly Streamlit app. Once uploaded, the system checks and saves the data, then cleans it by fixing column names and filling missing values. A detailed EDA (Exploratory Data Analysis) report is generated to help users understand the data

using visuals like scatter plots and heatmaps. Next, multiple machine learning models are trained and evaluated automatically using performance metrics like F1 Score and Accuracy. The best model is saved and can be downloaded for future use. Users can also load and compare this model with others to see which performs better, with results shown in a clear comparison table. Overall, the app offers a smooth, end-to-end machine learning workflow that’s easy to use—even for beginners.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 CONCLUSION

The project successfully developed a semi-automated machine learning workflow, focusing on data ingestion, preprocessing, exploratory data analysis (EDA), and model training. By automating dataset handling and EDA report generation, the system significantly reduced manual effort and improved data consistency. It streamlined model selection, allowing users to train and compare multiple machine learning models efficiently. The successful implementation demonstrated its capability in handling tabular datasets, optimizing model performance, and enabling model deployment and reuse. Overall, this work lays a strong foundation for future AutoML advancements, making machine learning more accessible and efficient.

6.2 FUTURE WORK

Future work aims to enhance system adaptability by integrating multi-modal data processing, including audio, video, and time-series analysis. Research will focus on improving model selection via meta-learning techniques and increasing transparency through explainable AI (XAI). Scalability improvements will explore distributed computing and efficient algorithms to manage large datasets. Further advancements will incorporate automated feature engineering and selection, boosting predictive accuracy while minimizing computational overhead. Additionally, integrating large language models (LLMs) could refine data preprocessing, generate intelligent EDA reports, and automate documentation, making the system more user-friendly and adaptable to evolving ML workflows.

REFERENCES

[1] Basu, A., & Roy, S. (2024)" Efficient AutoML with Meta-Features and Dynamic Configuration Adaptation" Data Mining and Knowledge Discovery, VOL.38, pp.181–203.

[2] Bother, T., Robroek, V., Gsteiger, R., Holzinger, X., Ma, X., Tözün, P., & Klimovic, A.(2025) "Modyn: Data-Centric Machine Learning Pipeline Orchestration" Proceedings of the ACM SIGMOD, vol. 45 ,pp. 1-30.

[3] Celik, P., Singh, R., & Vanschoren, J. (2023)" Online AutoML: An Adaptive AutoML Framework for Online Learning" Machine Learning,vol.112(6), pp.1897–1921, doi: 10.1007/s10994-022.

[4] Chen, L., Zhu, X., Liu, B., Yu, X., & Wang, S. (2024)"Online Class Incremental Learning With Adaptive Sample Memory Replay" IEEE Transactions on Neural Networks and Learning Systems,vol 35(1), pp.1–14.

[5] Du, B., & Shen, Y.-D. (2019)" Unsupervised Feature Selection with Adaptive Structure Learning" arXiv preprint arXiv:1504.00736,vol. 56, no. 3, pp. 1–34.

[6] Ghosh, A., Sankararaman, A., & Ramchandran, K. (2024)" Model Selection for Generic Contextual Bandits" IEEE Transactions on Information Theory, vol.70(1), pp.656–675.

[7] Gu, H., You, H., Cao, J., Yu, M., Fan, H., & Qian, S. (2024)"Large Language Models for Constructing and Optimizing Machine Learning Workflows: A Survey" arXiv preprint arXiv:2411.10478,vol. 38, pp. 181–203.

[8] Gutowska, A., Little, S., & McCarren, A. (2023)"Constructing a Meta-Learner for Unsupervised Anomaly Detection"IEEE Access, vol.11, pp.45815–45825.

[9] Han, X., Zhang, Z., Chen, H., Wu, H., & Li, M. (2023)"Dynamic Ensemble Selection Classification Algorithm Based on Window Over Imbalanced Drift Data Stream" Knowledge and Information Systems, vol.65(3), pp.1105–1128.

[10] Huang, M., Xie, X., & Yang, X. (2024)"Unsupervised Feature Selection via Controllable Adaptive Graph Learning and Discriminative Feature Learning" IEEE Transactions on Neural Networks and Learning Systems,vol. 35(11), pp.15600–15614.

[11] Huang, L., Dong, W., Wang, Y., Hao, Y., Singhal, S., Ma, S., Song, X. (2023)"Language Is Not All You Need: Aligning Perception with Language Models" arXiv preprint arXiv:2302.14045v2.vol.78(1),pp.1-8, doi: 10.1109/IJCNN60899.2024.10650909.

[12] Jiao, Y., Guo, D., Gong, D., & Chen, Q. (2024)" Dynamic Ensemble Selection for Imbalanced Data Streams With Concept Drift" IEEE Transactions on Neural Networks and Learning Systems, vol.35(1), pp.1278–1291.

[13] Kolbeinsson, A., Marla, L., & Yellepeddi, K. (2019)"Adaptive Model Selection Framework: An Application to Airline Pricing" Journal of Machine Learning Research, arXiv:1905.08874, vol. 618, pp. 191279.

[14] Liu, L. Wang, and H. Xu(2024) "Hybrid Meta-Learning and AutoML Framework for Structured and Unstructured Data," Information Sciences, vol. 658, p. 119279.

[15] Ming, W., Gong, Y., Wang, L., & Jin, Y. (2024)"Constrained Multi-Objective Optimization With Deep Reinforcement Learning Assisted Operator Selection" IEEE/CAA Journal of Automatica Sinica, vol.11(4), pp.919–931.

[16] Pahune, A., & Akhtar, Z. (2023)"Transitioning from MLOps to LLMOps: Navigating the Unique Challenges of Large Language Models" Dept. of Network & Computer Security, SUNY Binghamton, vol. 65, pp. 1105–1128.

[17] Ruder, L. M., Ramos, M., Choi, M., & Taylor, T. (2024)"Towards Efficient Meta-Learning Frameworks for Machine Learning Tasks" Journal of Artificial Intelligence Research, vol.58, pp.329–353.

[18] Sellars, A., Aviles-Rivero, A. I., & Schönlieb, C.-B. (2024). “LaplaceNet: A Hybrid Graph-Energy Neural Network for Deep Semi-supervised Classification” IEEE Transactions on Neural Networks and Learning Systems, vol.35(4), pp.5306–5318.

[19] Shen, Y., Wang, X., & Zhu, W. (2019)”A Self-Adaptive Parameter Selection Trajectory Prediction Approach via Hidden Markov Models” IEEE Transactions on Intelligent Transportation Systems,vol.16(1), pp.284–296.

[20] Vettoruzzo, R., Bouguelia, M. R., Vanschoren, J., Rognvaldsson, T., & Santosh, K. (2024)”Advances and Challenges in Meta-Learning: A Technical Review” IEEE Transactions on Pattern Analysis and Machine Intelligence, vol.46(7), pp.4763–4779.

[21] Wan, T., et al. (2022)”Meta-Learning with Task-Adaptive Selection” IEEE Transactions on Circuits and Systems for Video Technology, vol. 11, pp. 45815-45825, 2023, doi: 10.1109/ACCESS.3274113.

[22] Wang, W., Chen, J., Yu, J., Zhang, J., & Li, Y. (2024)”Deep Reinforcement Learning for Adaptive Model Selection in Autonomous Driving Systems” IEEE Transactions on Intelligent Transportation Systems, vol.23(5), pp.2157–2169.

[23] Wang, X., Liu, X., Liang, W., Zong, L., & Zhan, X. (2024)”Online Class - incremental Continual Learning with Maximum Entropy Memory Update” International Joint Conference on Neural Networks (IJCNN),vol.35,pp.15600-15614,doi: 10.1109/TNNLS.2023.3288734.

[24] Wang, X., Zhang, Z., Chen, H., Wu, H., & Li, M. (2024) “AutoML with Adaptive Sampling for Online Streaming Data” Knowledge and Information Systems, vol.65(2), pp.623–646.

[25] Wei, Y., Ma, X., Zhang, Y., & Liu, L. (2024)”Few-shot Class-Incremental Learning via Feature Space Augmentation and Self-supervised Adaptation” Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR, vol. 32, no. 12, pp. 8660-8673, doi: 10.1109/TCSVT.2022.3196092.

[26] Yang, B., Lu, J., & Xu, D. (2024)"A Meta-Learning Approach to Feature Selection and Hyperparameter Optimization" Journal of Machine Learning Research,vol. 40(7), pp.213–230.

[27] Yi, J., & He, Q. (2024) "Meta-Learning with Self-Adaptive Neural Architecture Search for Unsupervised Anomaly Detection" IEEE Transactions on Knowledge and Data Engineering, vol.36(10), pp.3050–3064.

[28] Yu, Z., & He, X. (2024) "A Survey on Model Selection Techniques in AutoML Systems" ACM Computing Surveys,vol. 56(3), pp.1–34.

[29] Yu, Z., & Li, Y. (2023) "Adaptive Sampling in Meta-Learning with Uncertainty Estimation" Neurocomputing,vol. 558, pp.126729.

[30] Yu, Z., & Xu, X. (2024) "Dynamic Model Selection for High-Dimensional Data: A Regularized Adaptive Learning Approach" Neural Networks and Learning Systems, vol.29(9), pp.3700–3710.

[31] Zhang, B., Li, J., & Lee, Y. (2024) "Adaptive Hyperparameter Selection via Reinforcement Learning" IEEE Transactions on Neural Networks and Learning Systems (ICNNS'24),vol.67, pp.1–10.

[32] Zhang, J., Li, Y., & Lee, Y. (2024) "Graph-Based Meta-Learning for Unsupervised Feature Selection" NeurIPS Proceedings,vol. 23, no. 5, pp. 2157-2169, doi: 10.1109/TITS.2023.3247809.

[33] Zhang, Z., & Liu, H. (2024) "Adaptive Learning with Meta-Features for Model Selection in Dynamic Environments" Machine Learning, vol.112(7), pp.1705–1722.

[34] Zhao, Y., & Xu, D. (2024) "Meta-Learning for Hyperparameter Optimization in AutoML: A Survey" International Journal of Data Science and Machine Learning, vol.7(2), pp.104–128.

[35] Zhang, F., & Wei, J. (2023). Reinforcement Learning-Based Feature Selection in Big Data Analytics. Proceedings of the 2023 IEEE Transaction on Big Data (BigData), San Diego, CA, USA,vol,27,pp. 4074-4083.

[36] Zhuang, J., Ma, L., & Xu, W. (2025). Neural Architecture Search for AutoML: Advances and Challenges. IEEE Transactions on Neural Networks and Learning Systems, vol.36(2), pp.283–295.

[37] Zhou, T., Wang, H., & Lu, M. (2024) “A Hybrid Meta-Learning Approach for AutoML in Time-Series Forecasting” Expert Systems with Applications,vol. 75,pp. 327–338.